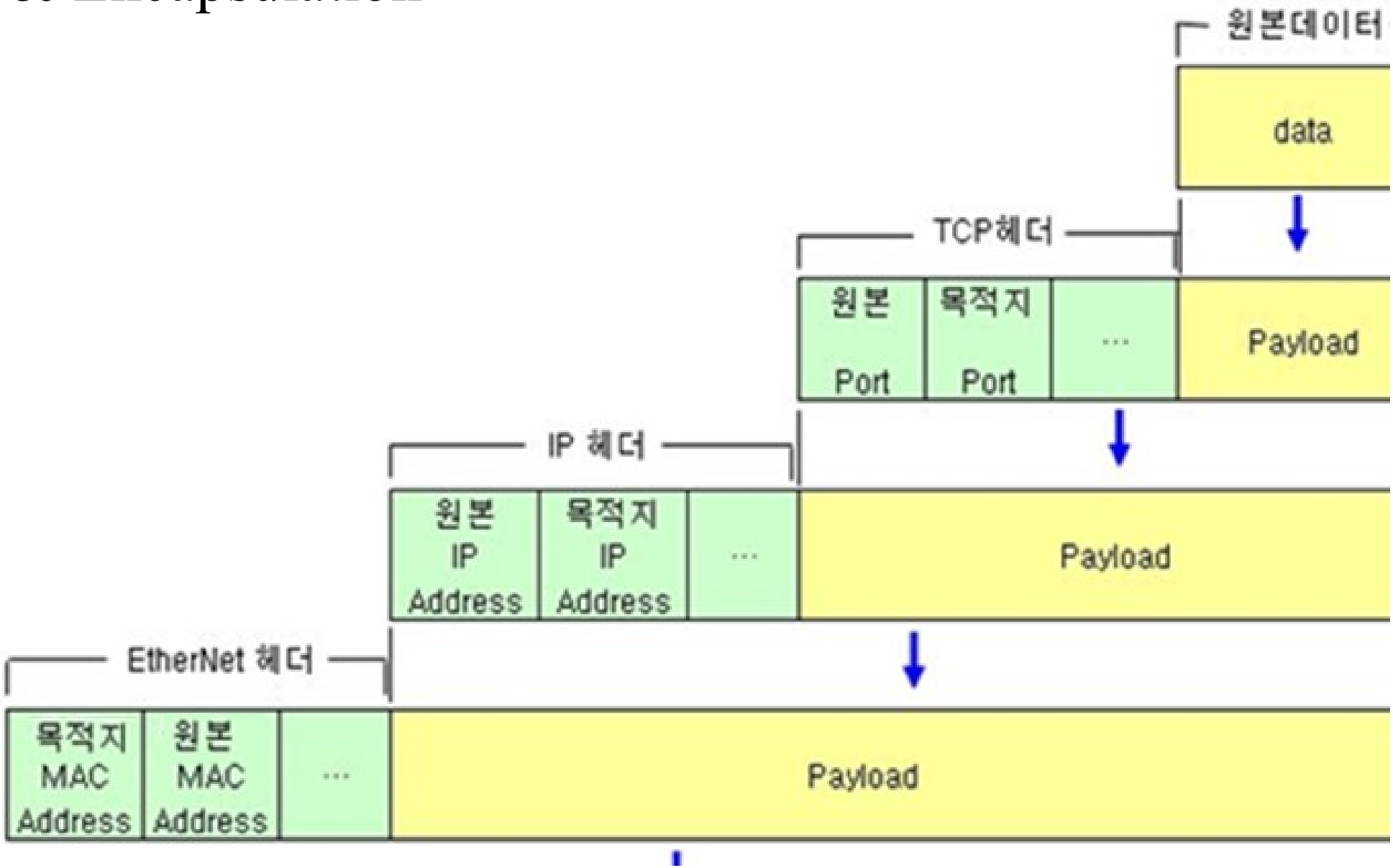
TCP 단편화

Payload & Encapsulation



UDP

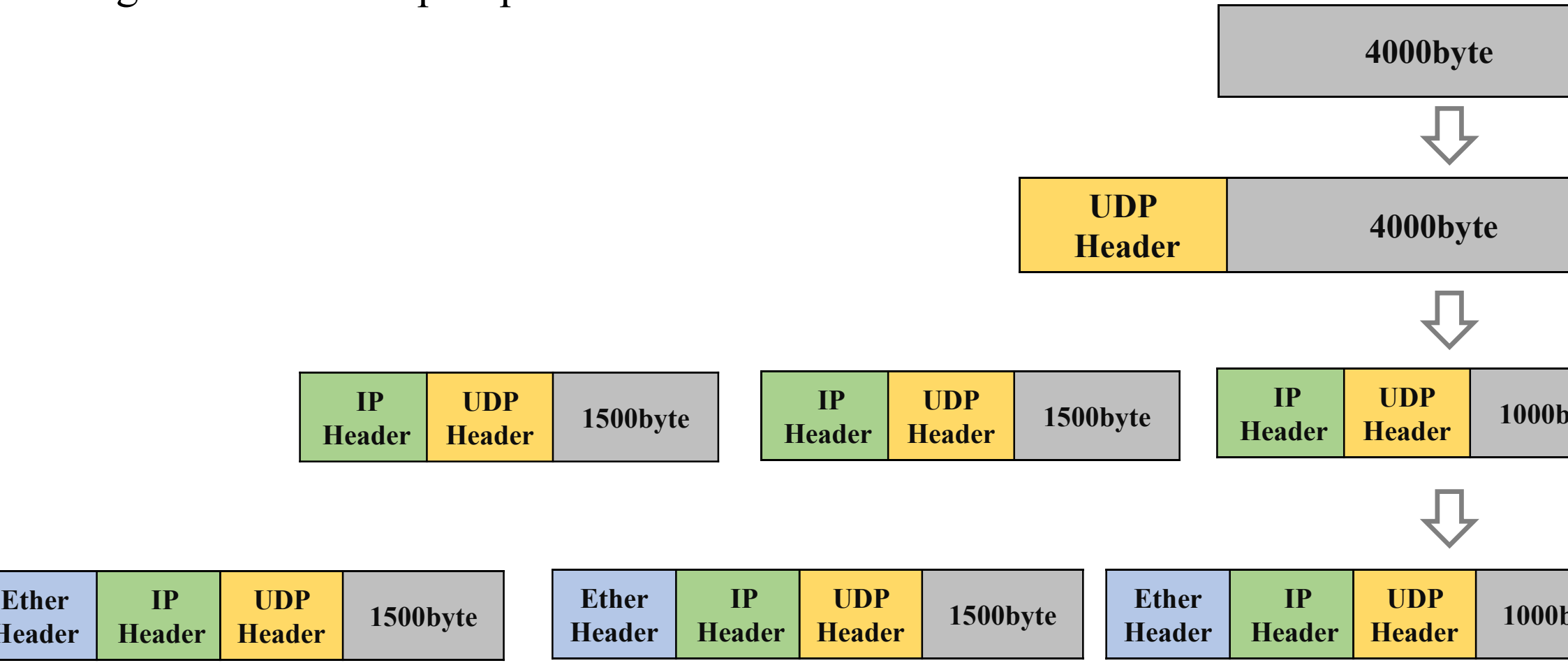
- Message-Oriented Transport Protocol
- 상위계층에서 payload 된 데이터를 단편화하지 않음
- 데이터 전송이 완료되지 전까지는 다른 데이터 흐름에 영향을 받지 않음

TCP

- Stream-Oriented Transport Protocol
- 상위계층에서 payload 된 데이터를 단편화
- 단편화를 통해 다수의 데이터들과 네트워크 자원을

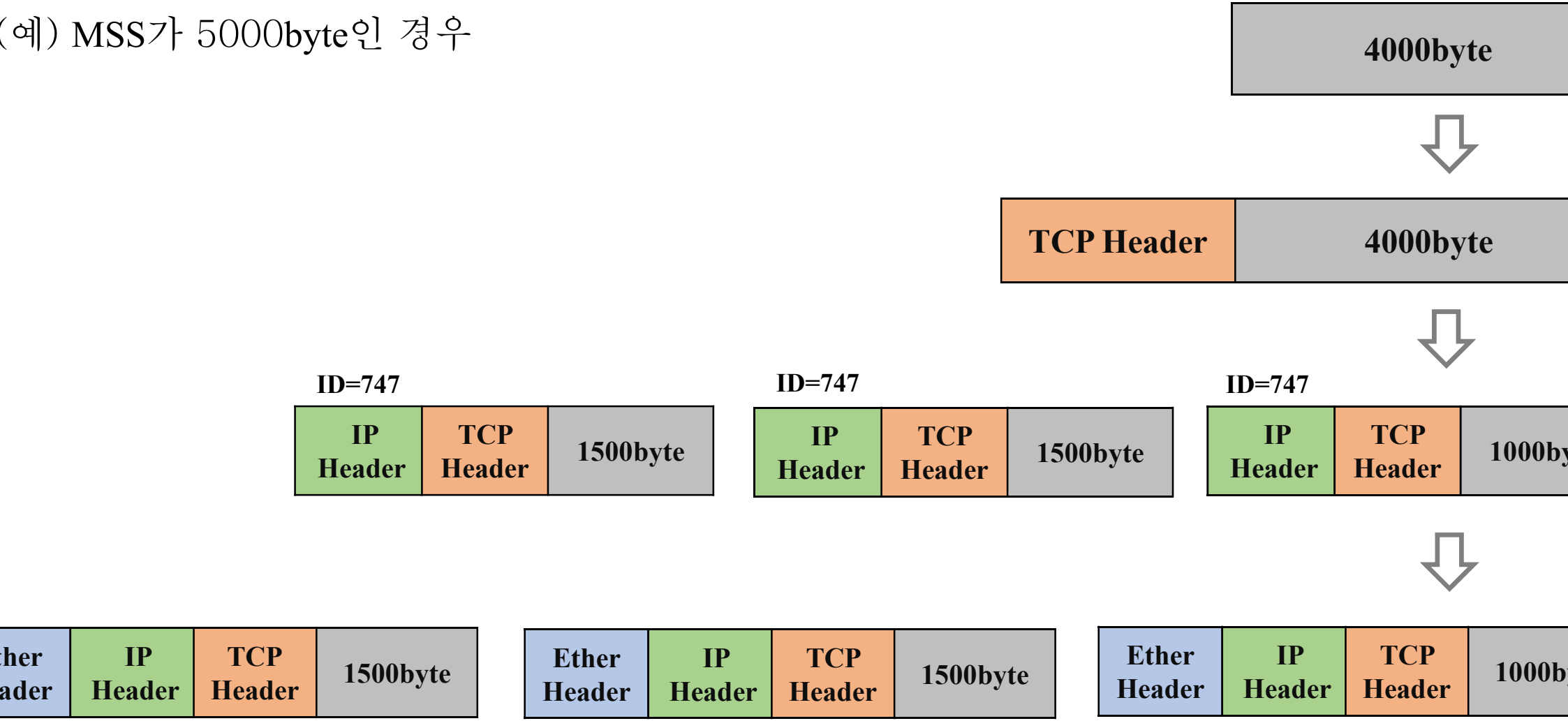
UDP 기반의 애플리케이션 데이터 전송

Message-oriented transport protocol



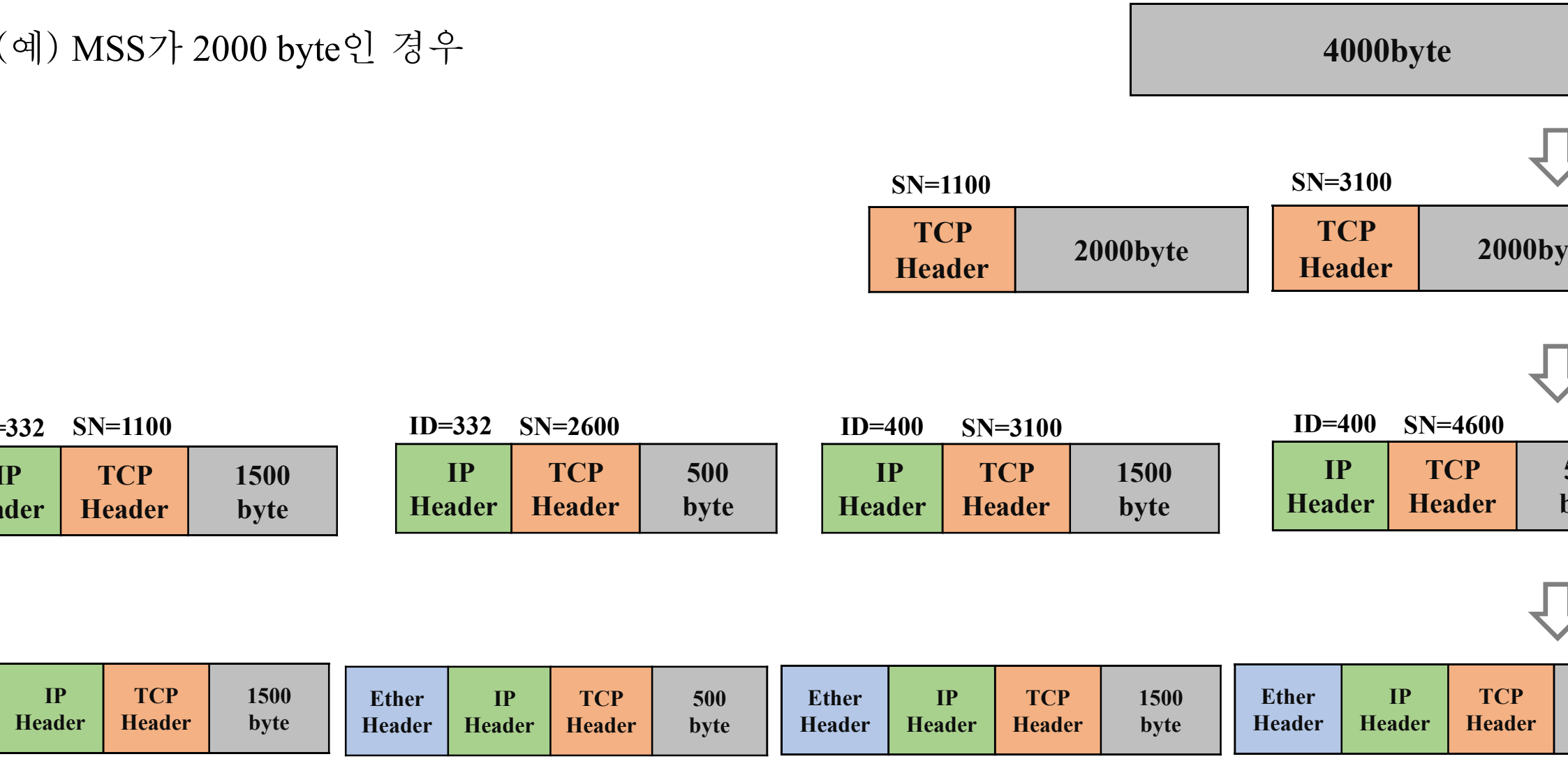
TCP 기반의 애플리케이션 데이터 전송

(예) MSS가 5000byte인 경우



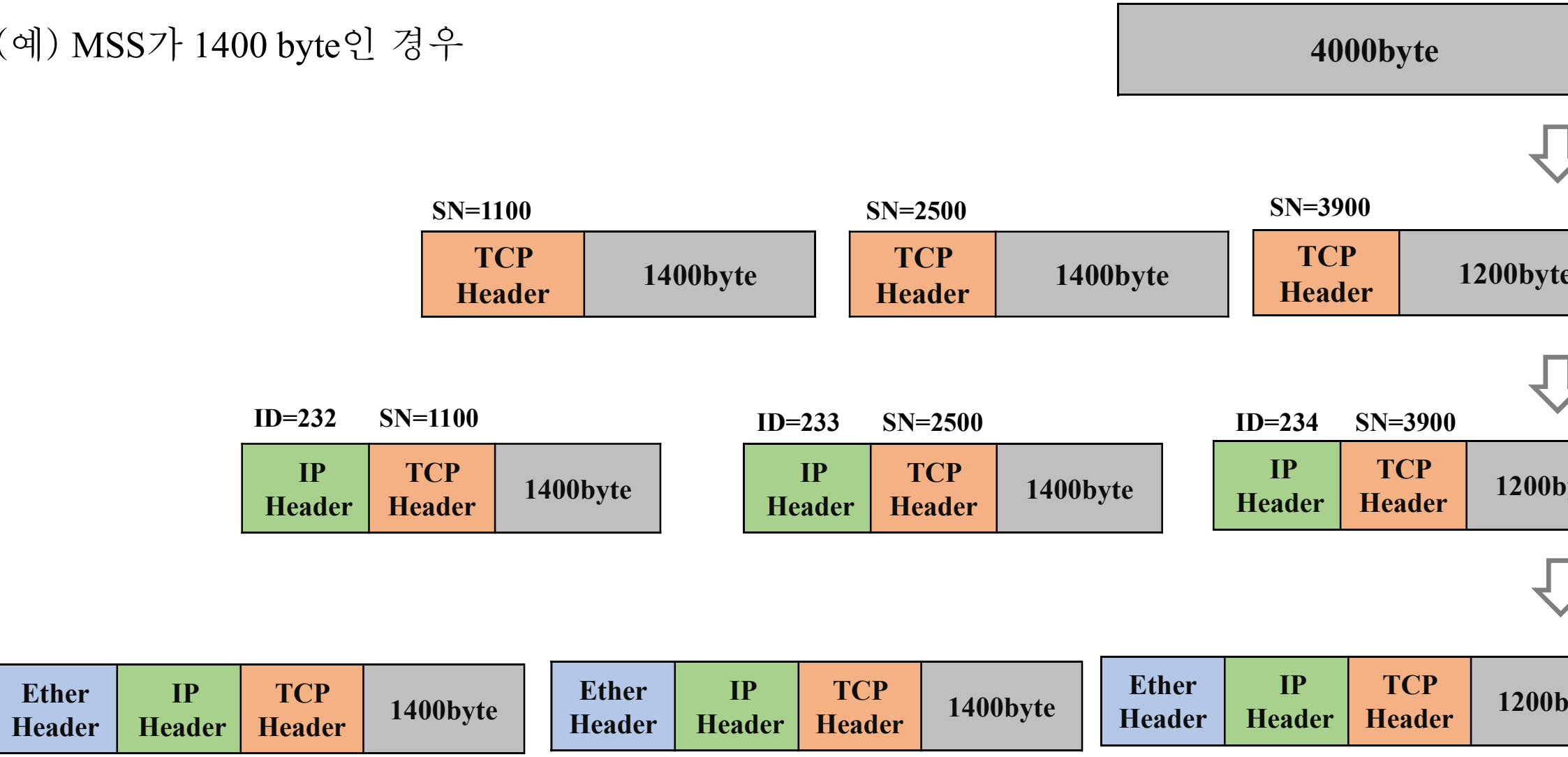
TCP 기반의 애플리케이션 데이터 전송

(예) MSS가 2000 byte인 경우



TCP 기반의 애플리케이션 데이터 전송

(예) MSS가 1400 byte인 경우

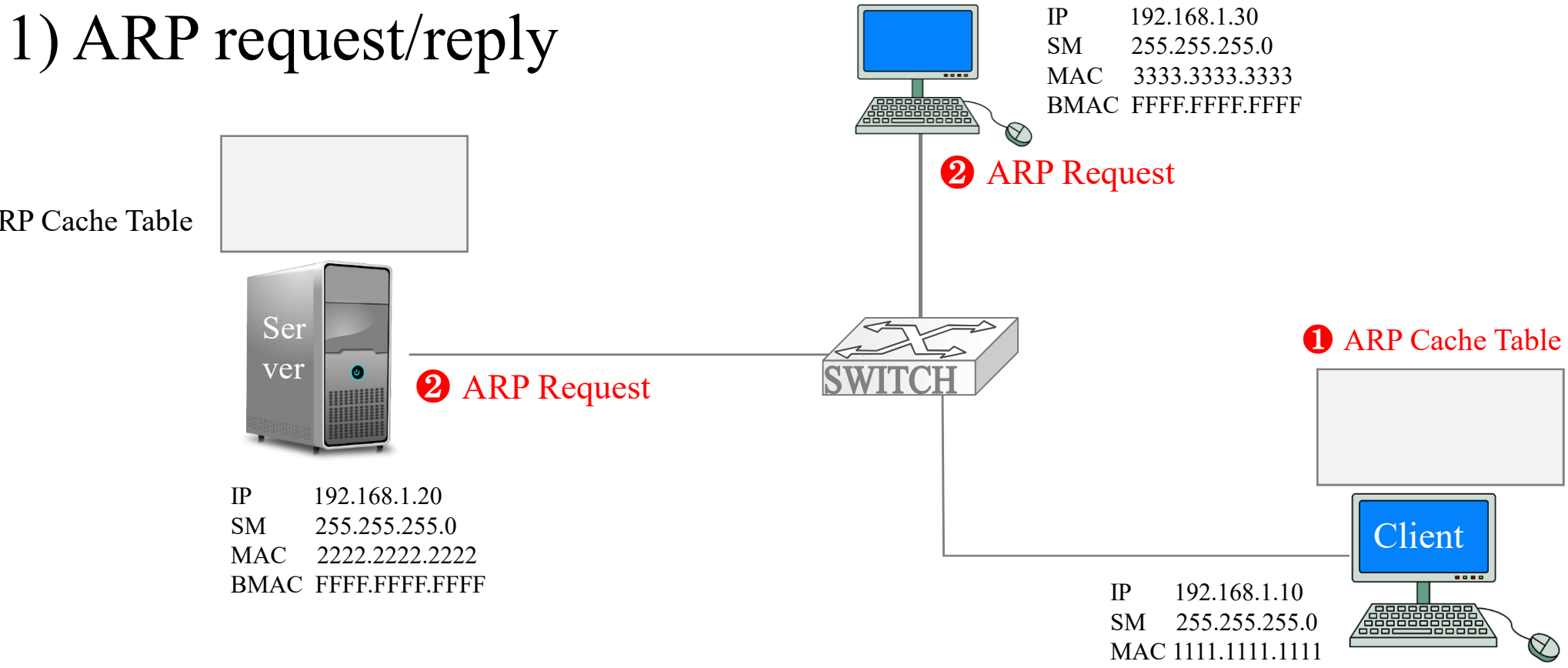


ARP

ARP Cache table update

- ARP request 패킷을 수신한 시스템들은 송신 시스템의 IP와 MAC주소를 자신의 ARP 캐시 엔트리에 저장
  - 이 경우 기존 캐시 정보의 다른 엔트리를 지울 수 있음
- 기존 캐시 정보의 다른 엔트리를 지울 수 있으므로 ARP 캐시에 송신 시스템의 IP 주소가 <u>이미 존재하는 시스템만이 ARP 캐시 엔트리를 갱신하고</u>, <u>ARP 캐시에 송신 시스템의 IP 주소가 없는 시스템은 ARP 요청 패킷을 무시</u>
- 시스템들은 ARP 요청 브로드캐스트에 있는 IP주소와 MAC 주소에 대해 자신의 <u>엔트리에 속해 있지 않은 정보는 무시하고 기존에 있는 정보에 대해서만 엔트리를 갱신해 ARP 캐시 엔트리의 무차별적인 갱신을 방지</u>

1) ARP request/reply



ARP Request

수MAC	송MAC	Protocol	ARP Header	
FFFF.FFFF.FFFF	1111.1111.1111	0x0806	1111.1111.1111 /192.168.1.10 **0000.0000.0000/192.168.1.20**	송신지 수신지

수MAC	송MAC	송IP	수IP	송Port	수Port	전송데이타
????	1111. 1111.1111	192.168.1.10	**192.168.1.20**	50030	80	Get /www.test.com

1) ARP request/reply

	ARP Header	Protocol	송MAC	수MAC
송신지 수신지	2222.2222.2222 /192.168.1.20 1111.1111.1111 /192.168.1.10	0x0806	2222.2222.2222	1111.1111.1111

ARP Reply

RP Cache Table

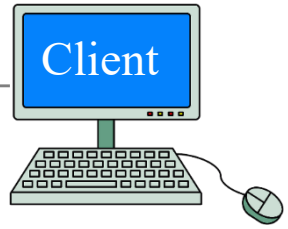
❶ 192.168.1.10 1111.1111.1111

ARP Cache Table

❸ 192.168.1.20 2222.2222.2222

IP 192.168.1.20
SM 255.255.255.0
MAC 2222.2222.2222
BMAC FFFF.FFFF.FFFF

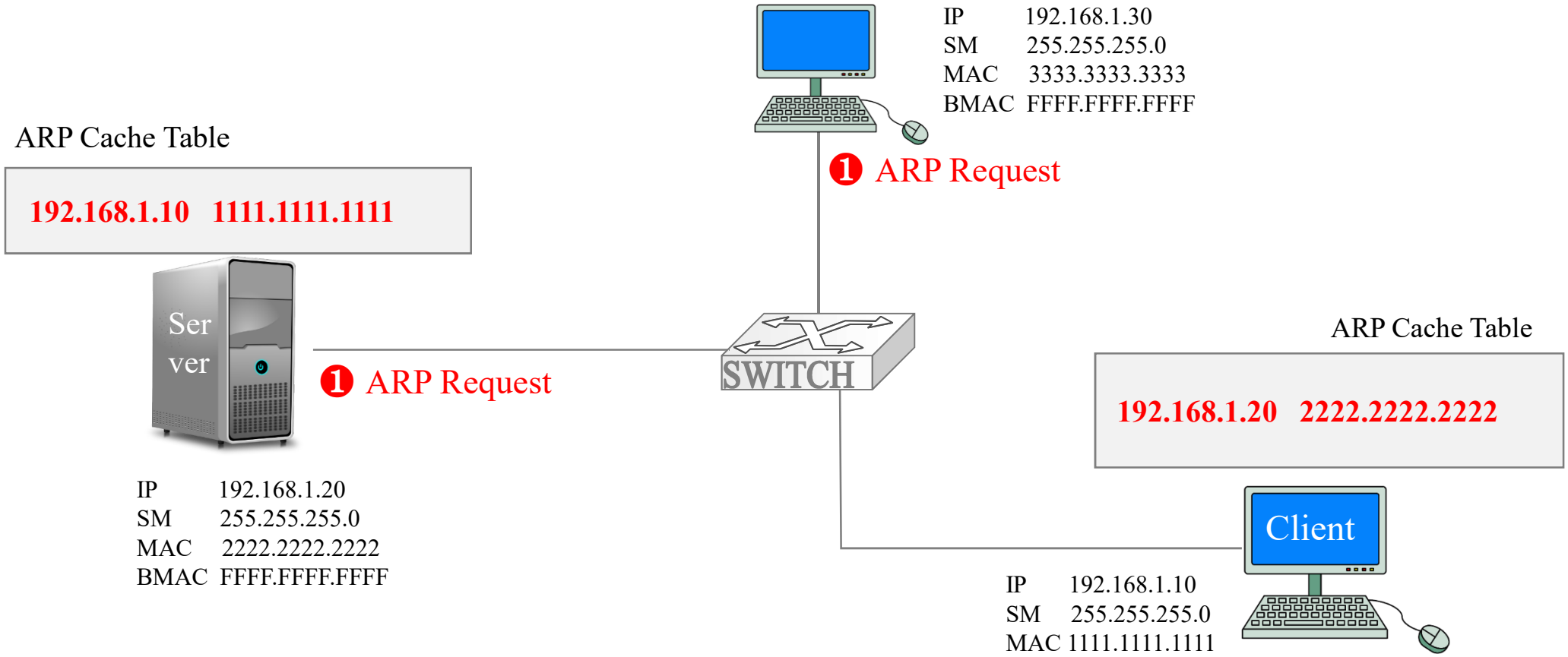
❷ ARP Reply



IP 192.168.1.10
SM 255.255.255.0
MAC1111.1111.1111

수MAC	송MAC	송IP	수IP	송Port	수Port	전송데이타
222.2222.2222	1111. 1111.1111	192.168.1.10	192.168.1.20	50030	80	Get /www.test.com

2) ARP request (Unicast)



ARP Request

수MAC	송MAC	Protocol	ARP Header	
2222.2222.2222	1111.1111.1111	0x0806	1111.1111.1111 /192.168.1.10 2222.2222.2222 /192.168.1.20	송신자 수신자

ARP reply가 전송되지 않으면 ARP 캐시테이블에 해당 IP와 MAC 주소 삭제

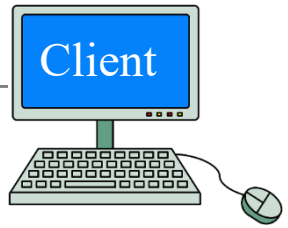
ARP Cache Table

192.168.1.10 1111.1111.1111

IP 192.168.1.20
SM 255.255.255.0
MAC 2222.2222.2222
BMAC FFFF.FFFF.FFFF

ARP Cache Table

~~192.168.1.20 2222.2222.2222~~



IP 192.168.1.10
SM 255.255.255.0
MAC1111.1111.1111

수MAC	송MAC	송IP	수IP	송Port	수Port	전송데이타
222.2222.2222	1111. 1111.1111	192.168.1.10	**192.168.1.20**	50030	80	Get /www.test.com

arppoison.pcapng

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

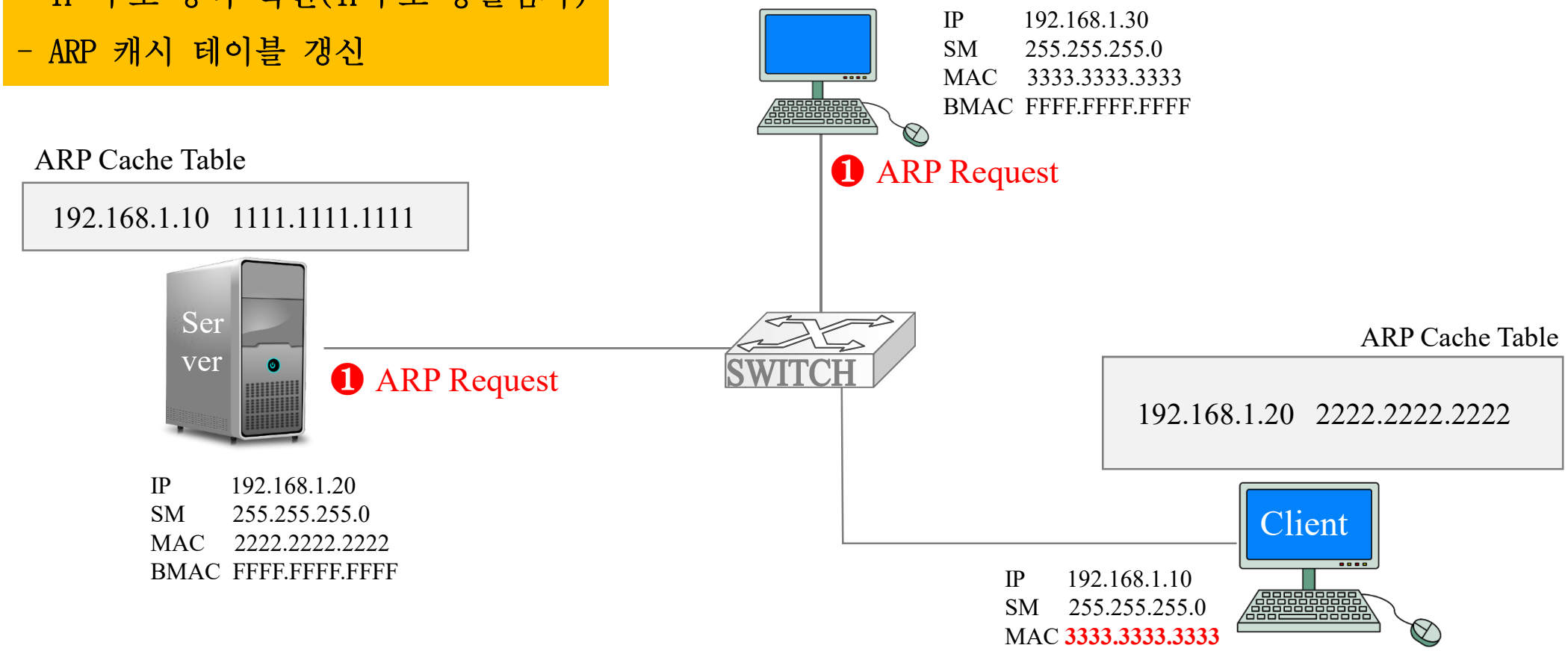
arp

No.	Time	Source	Destination	Protocol
54	4.171500	00:25:b3:bf:91:ee	00:21:70:c0:56:f0	ARP
55	0.000053	00:21:70:c0:56:f0	00:25:b3:bf:91:ee	ARP
56	0.000013	00:25:b3:bf:91:ee	00:21:70:c0:56:f0	ARP
165	3.468946	00:25:b3:bf:91:ee	ff:ff:ff:ff:ff:ff	ARP

```
> Frame 54: 60 bytes on wire (480 bits), 60 bytes captured (480 bits) on
v Ethernet II, Src: 00:25:b3:bf:91:ee, Dst: 00:21:70:c0:56:f0
  > Destination: 00:21:70:c0:56:f0
  > Source: 00:25:b3:bf:91:ee
    Type: ARP (0x0806)
    Padding: 000000000000000000000000000000000000
v Address Resolution Protocol (request)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: request (1)
    Sender MAC address: 00:25:b3:bf:91:ee
    Sender IP address: 172.16.0.1
    Target MAC address: 00:00:00:00:00:00
    Target IP address: 172.16.0.107
```

3) gARP(gratuitous ARP)

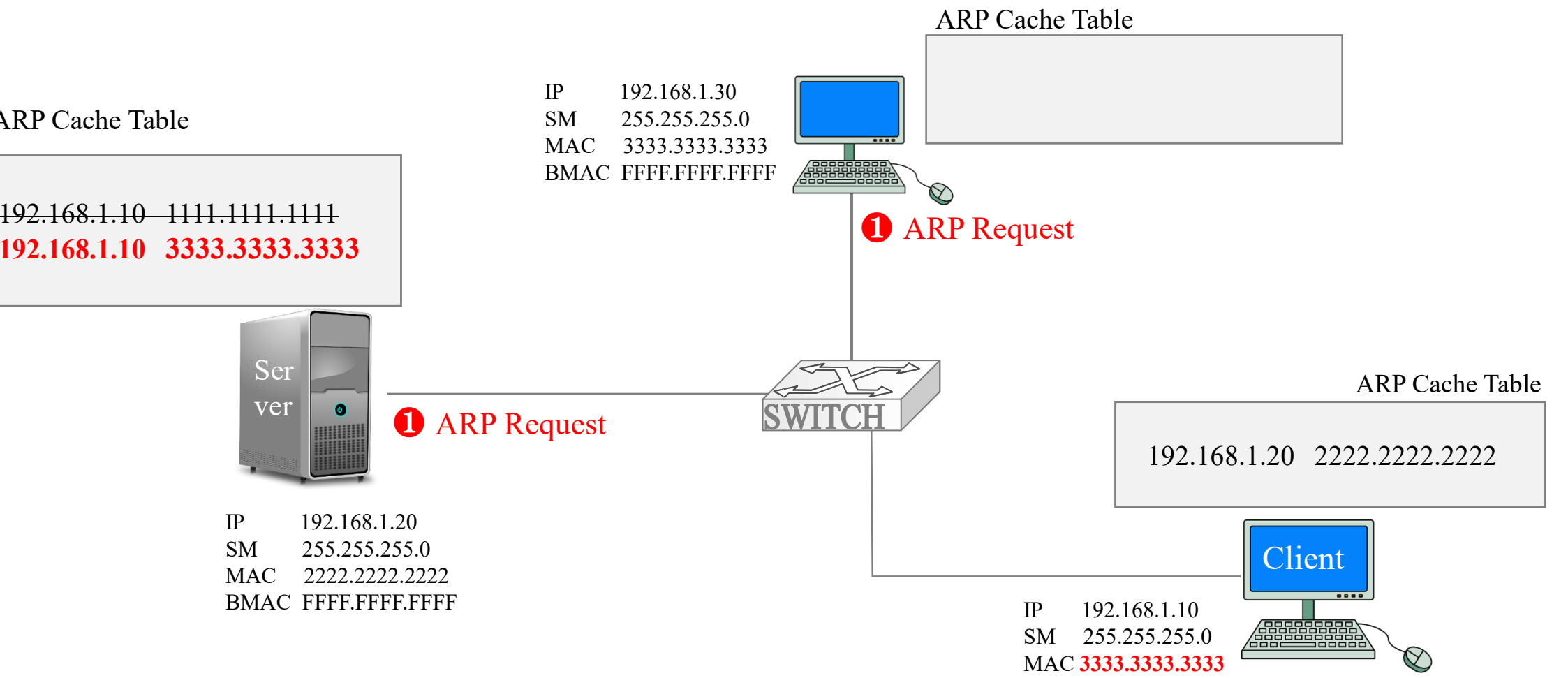
- IP 주소 중복 확인(IP주소 충돌검사)
- ARP 캐시 테이블 갱신



ARP Request

수MAC	송MAC	Protocol	ARP Header
FFFF.FFFF.FFFF	3333.3333.3333	0x0806	3333.3333.3333 /192.168.1.10 0000.0000.0000/192.168.1.10

송
수



ARP Request

수MAC	송MAC	Protocol	ARP Header
FFFF.FFFF.FFFF	3333.3333.3333	0x0806	3333.3333.3333 /192.168.1.10 0000.0000.0000/192.168.1.10

arp

No.	Time	Source	Destination	Protocol	Length	Info
160	0.001152	a4:7b:2c:20:2f:67	ff:ff:ff:ff:ff:ff	ARP	60	Who has 175.196.82.72
161	0.014854	a4:7b:2c:20:2f:67	ff:ff:ff:ff:ff:ff	ARP	60	Who has 175.196.82.16
173	0.572971	a4:7b:2c:20:2f:67	ff:ff:ff:ff:ff:ff	ARP	60	Who has 221.148.117.18
179	0.159985	98:fa:9b:0f:35:53	ff:ff:ff:ff:ff:ff	ARP	42	ARP Announcement for 1
181	0.246186	a4:7b:2c:20:2f:67	ff:ff:ff:ff:ff:ff	ARP	60	Who has 175.196.82.13
182	0.000387	a4:7b:2c:20:2f:67	ff:ff:ff:ff:ff:ff	ARP	60	Who has 175.196.82.118
183	0.000620	a4:7b:2c:20:2f:67	ff:ff:ff:ff:ff:ff	ARP	60	Who has 175.196.82.110

```
> Frame 179: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\NPF
v Ethernet II, Src: 98:fa:9b:0f:35:53, Dst: ff:ff:ff:ff:ff:ff
  > Destination: ff:ff:ff:ff:ff:ff
  > Source: 98:fa:9b:0f:35:53
    Type: ARP (0x0806)
v Address Resolution Protocol (ARP Announcement)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: request (1)
    [Is gratuitous: True]
```

<<Gratuitous ARP 전송 횟수를 줄이는 방법 >>

① 시작 -> 실행 -> regedit(또는 regedt32)

② **HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Services\Tcpip**

\Parameters 로 이동

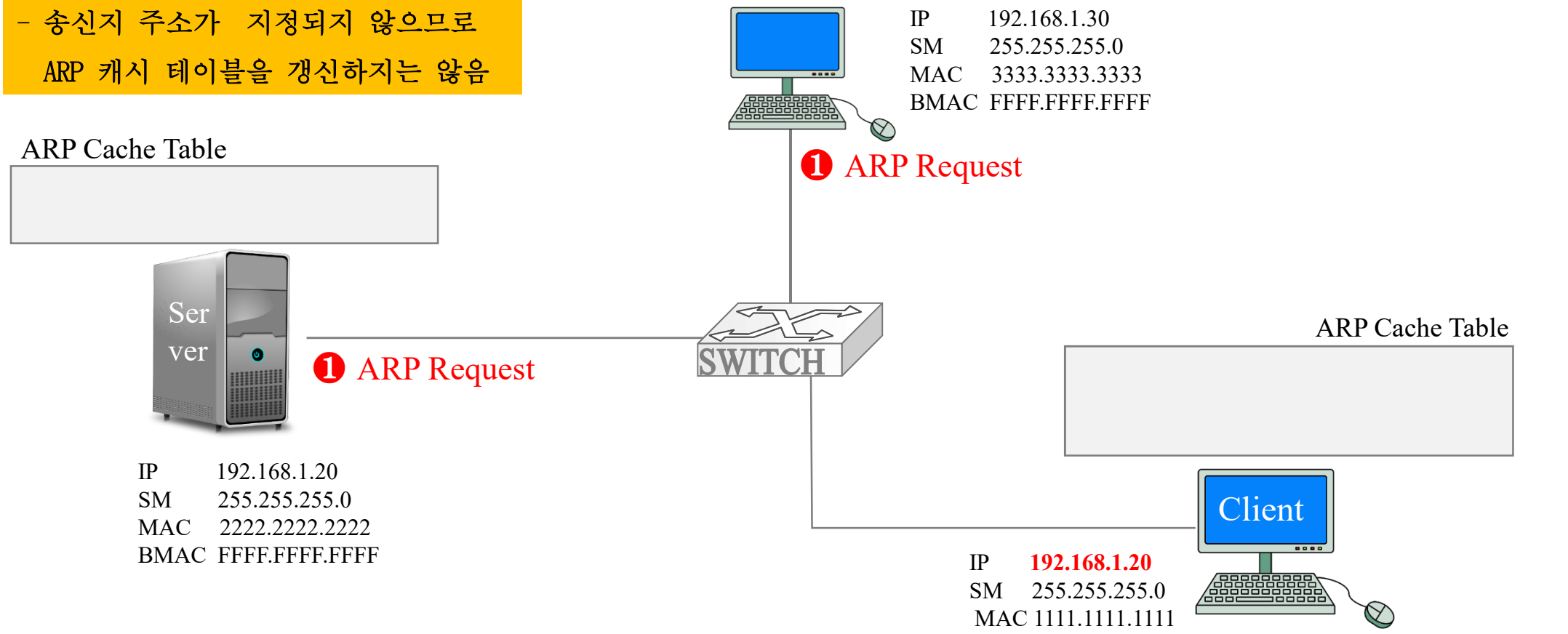
③ DWORD 값으로 **ArpRetryCount** 생성

➔ 값의 범위는 **0 ~ 3**

➔ 0일 경우 GARP를 보내지 않음

4) ARP Probe

- IP 주소 중복 확인(IP주소 충돌검사)
- 송신지 주소가 지정되지 않으므로 ARP 캐시 테이블을 갱신하지는 않음



ARP Request

수MAC	송MAC	Protocol	ARP Header
FFFF.FFFF.FFFF	1111.1111.1111	0x0806	1111.1111.1111 / 0.0.0.0 0000.0000.0000 /192.168.1.20

송신지
수신지

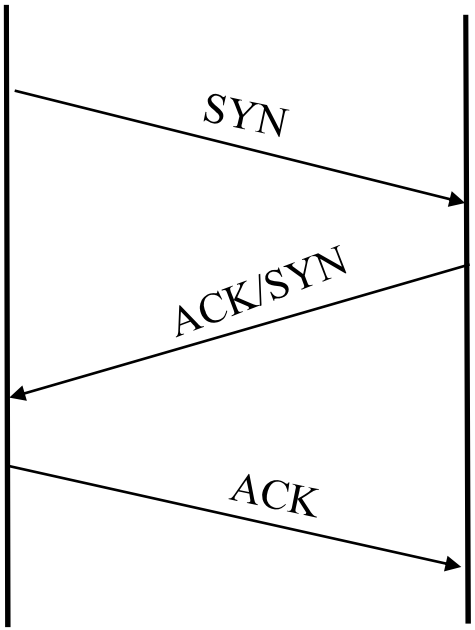
arp

No.	Time	Source	Destination	Protocol	Length	Info
67	0.004147	a4:7b:2c:20:2f:67	ff:ff:ff:ff:ff:ff	ARP	60	Who has 175.196.82.
88	0.509857	a4:7b:2c:20:2f:67	ff:ff:ff:ff:ff:ff	ARP	60	Who has 175.196.82.
93	0.108607	a4:7b:2c:20:2f:67	ff:ff:ff:ff:ff:ff	ARP	60	Who has 175.196.82.
101	0.115649	98:fa:9b:0f:35:53	ff:ff:ff:ff:ff:ff	ARP	42	Who has 172.16.7.26
140	1.009734	98:fa:9b:0f:35:53	ff:ff:ff:ff:ff:ff	ARP	42	Who has 172.16.7.26
159	0.239439	a4:7b:2c:20:2f:67	ff:ff:ff:ff:ff:ff	ARP	60	Who has 175.196.82.
160	0.001152	a4:7b:2c:20:2f:67	ff:ff:ff:ff:ff:ff	ARP	60	Who has 175.196.82

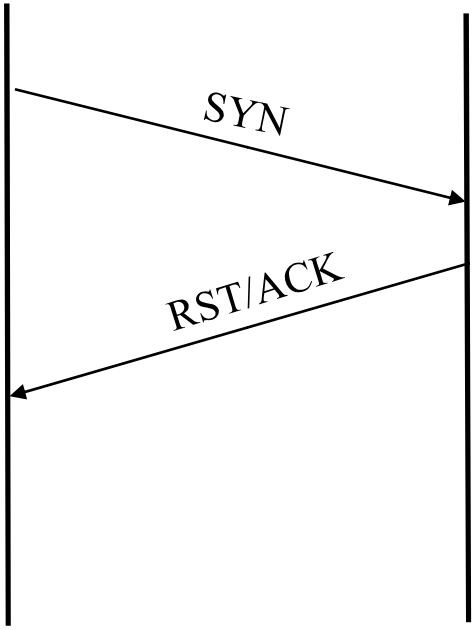
```
> Frame 101: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device\
> Ethernet II, Src: 98:fa:9b:0f:35:53, Dst: ff:ff:ff:ff:ff:ff
v Address Resolution Protocol (ARP Probe)
    Hardware type: Ethernet (1)
    Protocol type: IPv4 (0x0800)
    Hardware size: 6
    Protocol size: 4
    Opcode: request (1)
    [Is probe: True]
```

서비스 활성화 여부 점검

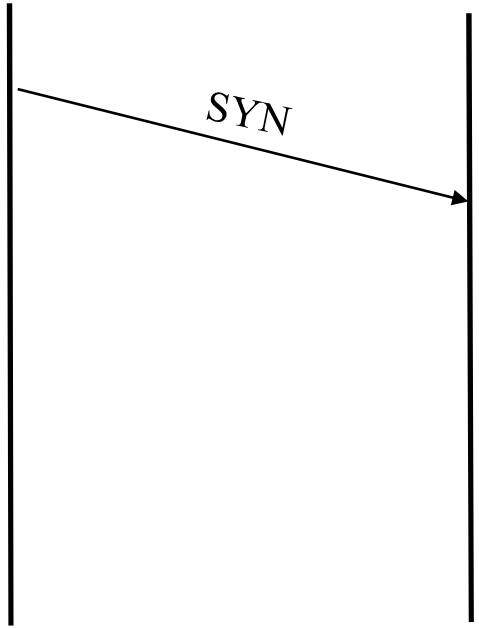
1) TCP 서비스 활성화 여부 점검



<<열린 포트인 경우 >>

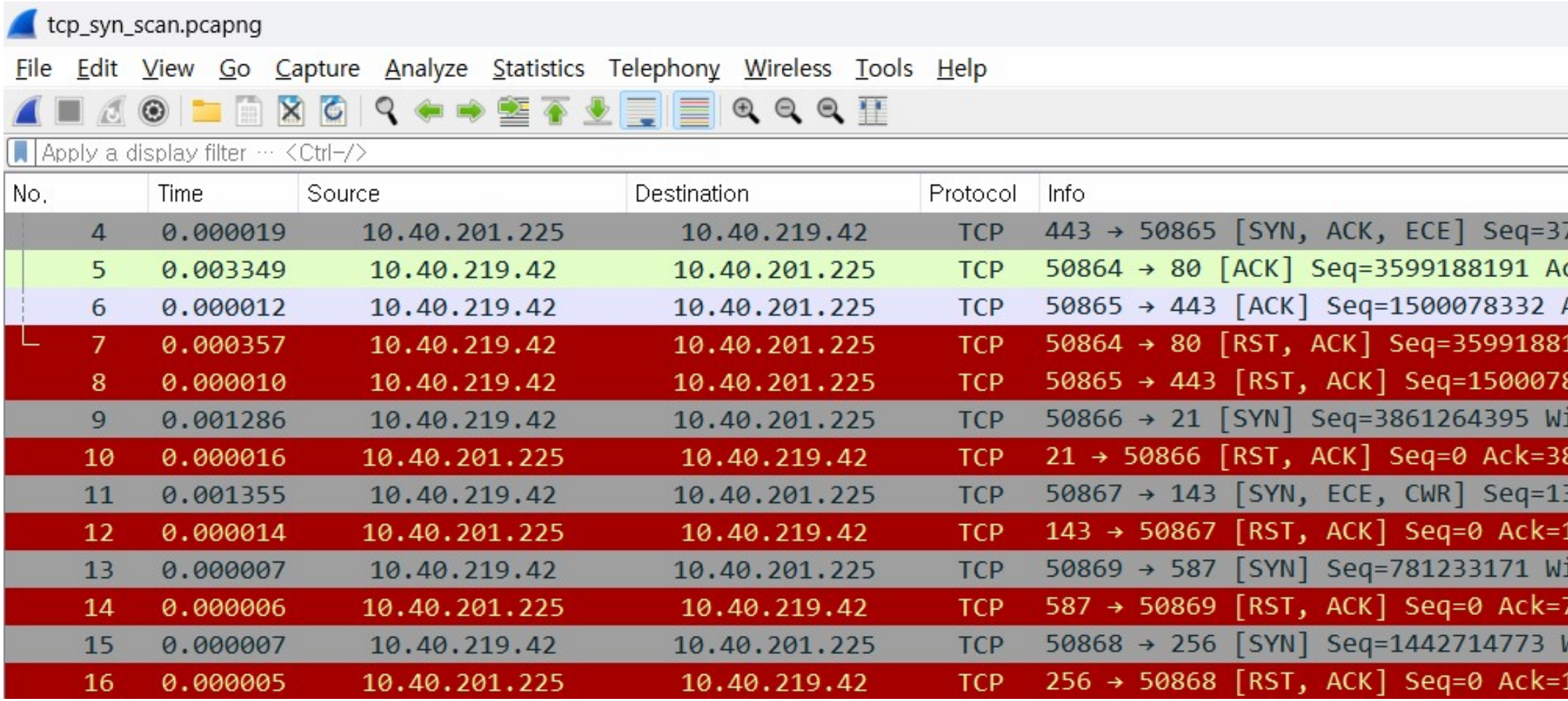


<<닫힌 포트인 경우 >>



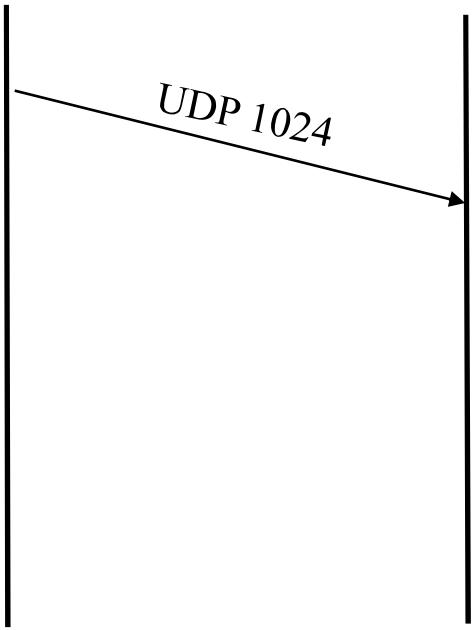
<<차단정책으로 차단되어 있는 경우>>

1) TCP 서비스 활성화 여부 점검

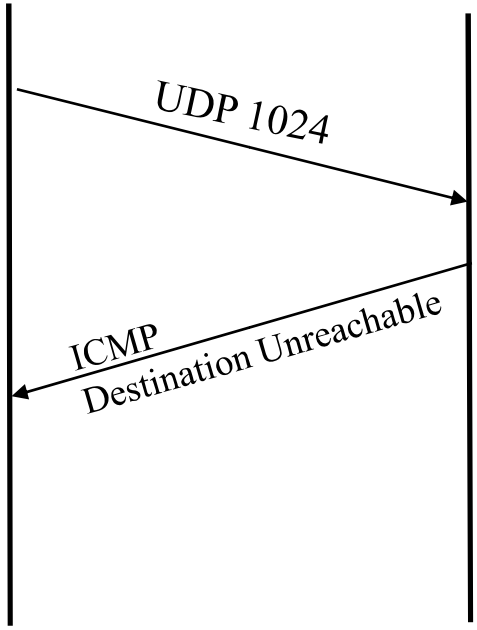


No.	Time	Source	Destination	Protocol	Info
4	0.000019	10.40.201.225	10.40.219.42	TCP	443 → 50865 [SYN, ACK, ECE] Seq=37
5	0.003349	10.40.219.42	10.40.201.225	TCP	50864 → 80 [ACK] Seq=3599188191 Ac
6	0.000012	10.40.219.42	10.40.201.225	TCP	50865 → 443 [ACK] Seq=1500078332 A
7	0.000357	10.40.219.42	10.40.201.225	TCP	50864 → 80 [RST, ACK] Seq=3599188
8	0.000010	10.40.219.42	10.40.201.225	TCP	50865 → 443 [RST, ACK] Seq=150007
9	0.001286	10.40.219.42	10.40.201.225	TCP	50866 → 21 [SYN] Seq=3861264395 Wi
10	0.000016	10.40.201.225	10.40.219.42	TCP	21 → 50866 [RST, ACK] Seq=0 Ack=38
11	0.001355	10.40.219.42	10.40.201.225	TCP	50867 → 143 [SYN, ECE, CWR] Seq=13
12	0.000014	10.40.201.225	10.40.219.42	TCP	143 → 50867 [RST, ACK] Seq=0 Ack=1
13	0.000007	10.40.219.42	10.40.201.225	TCP	50869 → 587 [SYN] Seq=781233171 Wi
14	0.000006	10.40.201.225	10.40.219.42	TCP	587 → 50869 [RST, ACK] Seq=0 Ack=7
15	0.000007	10.40.219.42	10.40.201.225	TCP	50868 → 256 [SYN] Seq=1442714773 W
16	0.000005	10.40.201.225	10.40.219.42	TCP	256 → 50868 [RST, ACK] Seq=0 Ack=1

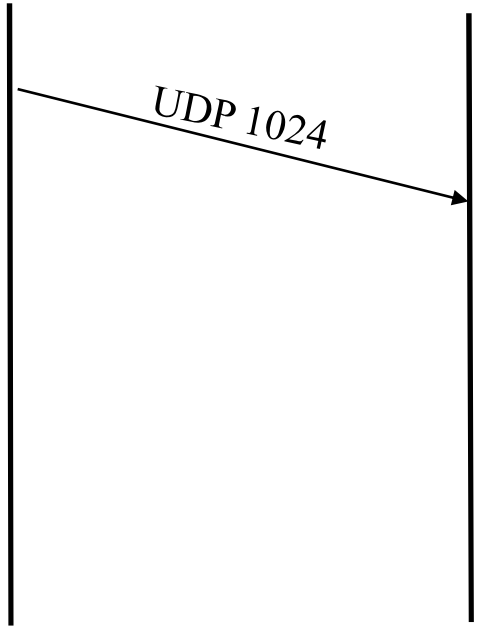
2) UDP 서비스 활성화/비활성화 여부 점검



<<열린 포트인 경우 >>

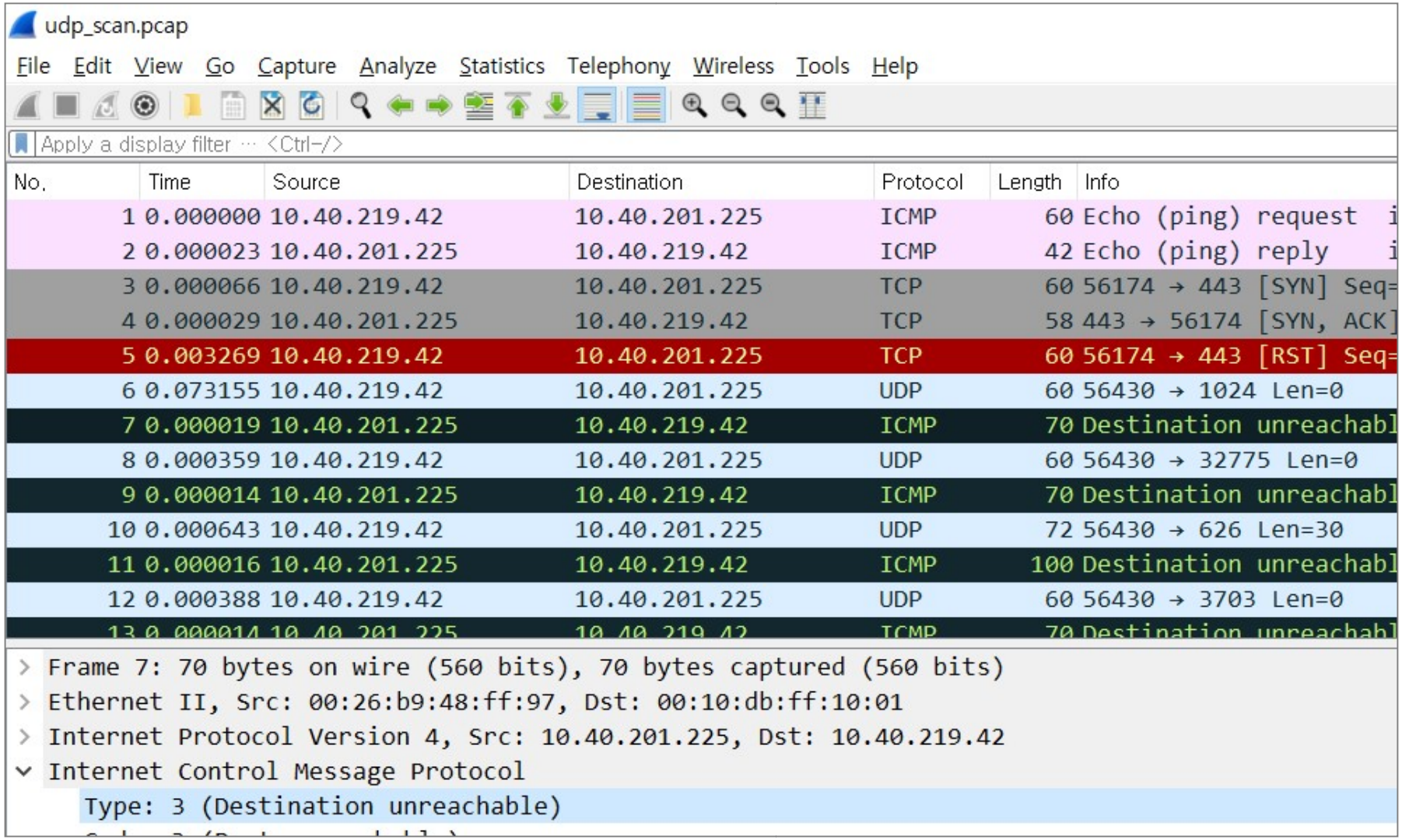


<<닫힌 포트인 경우 >>

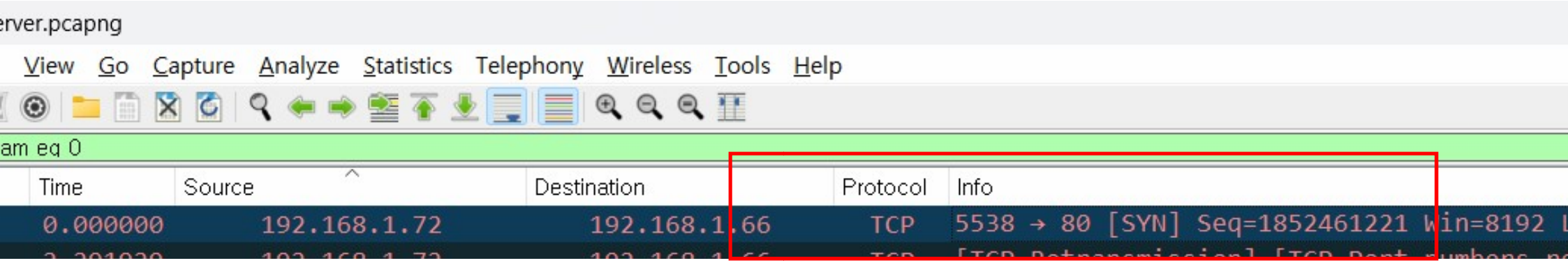


<<차단정책으로 차단되어 있는 경우>>

2) UDP 서비스 활성화/비활성화 여부 점검

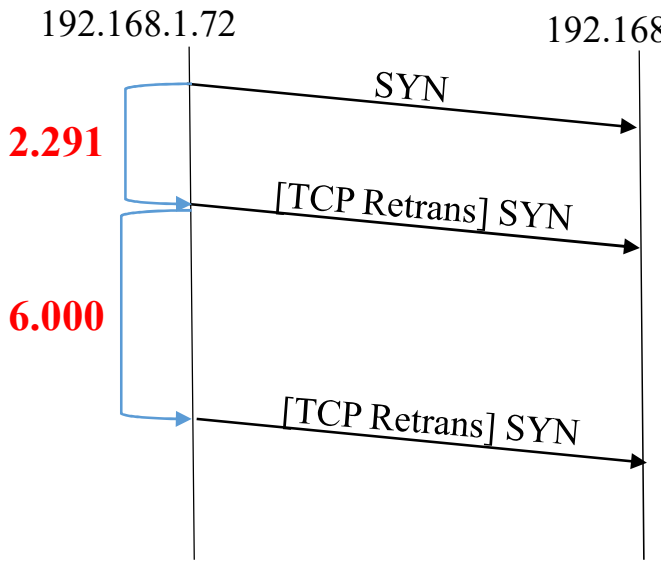


서버 응답 부재

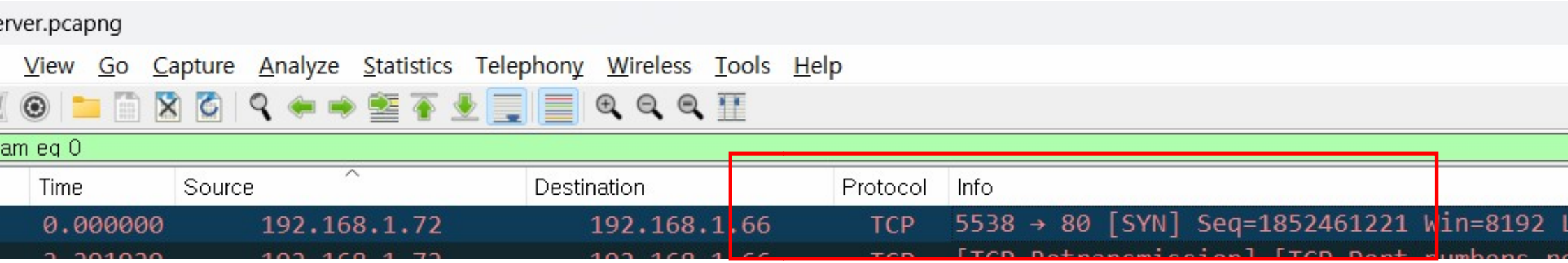


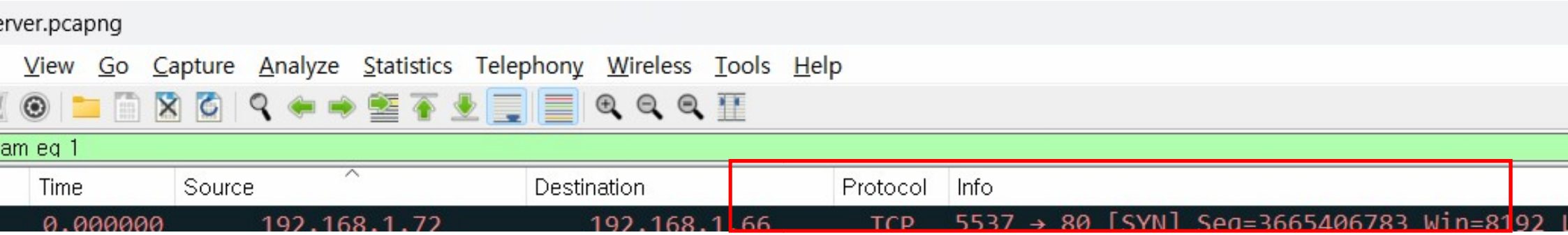
서버가 TCP 연결시도에 응답하지 않는 경우
- SYN가 서버에 도착하지 않은 경우
- 방화벽이 SYN 패킷을 폐기할 경우
- 서버 방화벽이 포트 접근을 차단한 경우
- SYN/ACK 가 클라이언트에 도달하지 않은 경우

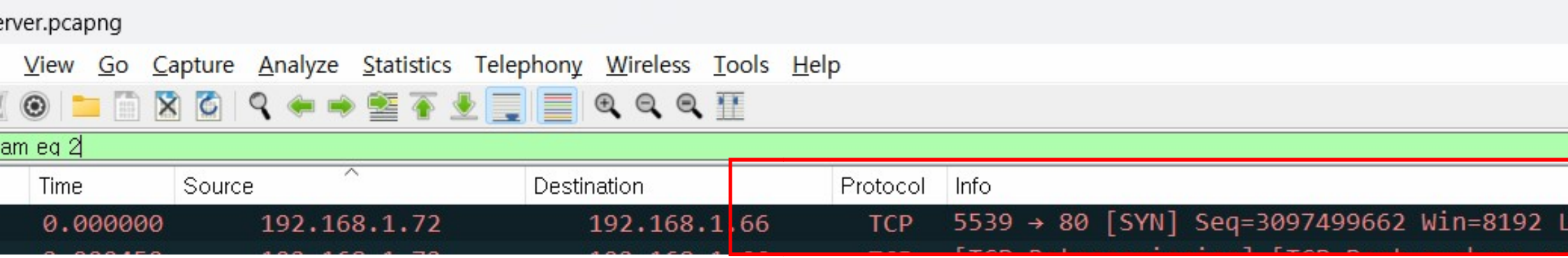
SYN 패킷에 대한 SYN/ACK 응답을 받지 못할 경우
- 클라이언트 측에서 SYN/ACK를 볼 수 없다면 서버쪽에서 SYN/ACK를 보냈는지를 확인
- SYN/ACK가 서버쪽에서 보내졌다면 경로 상 장치들 중에서 패킷을 폐기한 것임
- 폐기 지점을 찾을 때까지 캡처 지점을 클라이언트에 가까운 쪽으로 이동하면서 점검



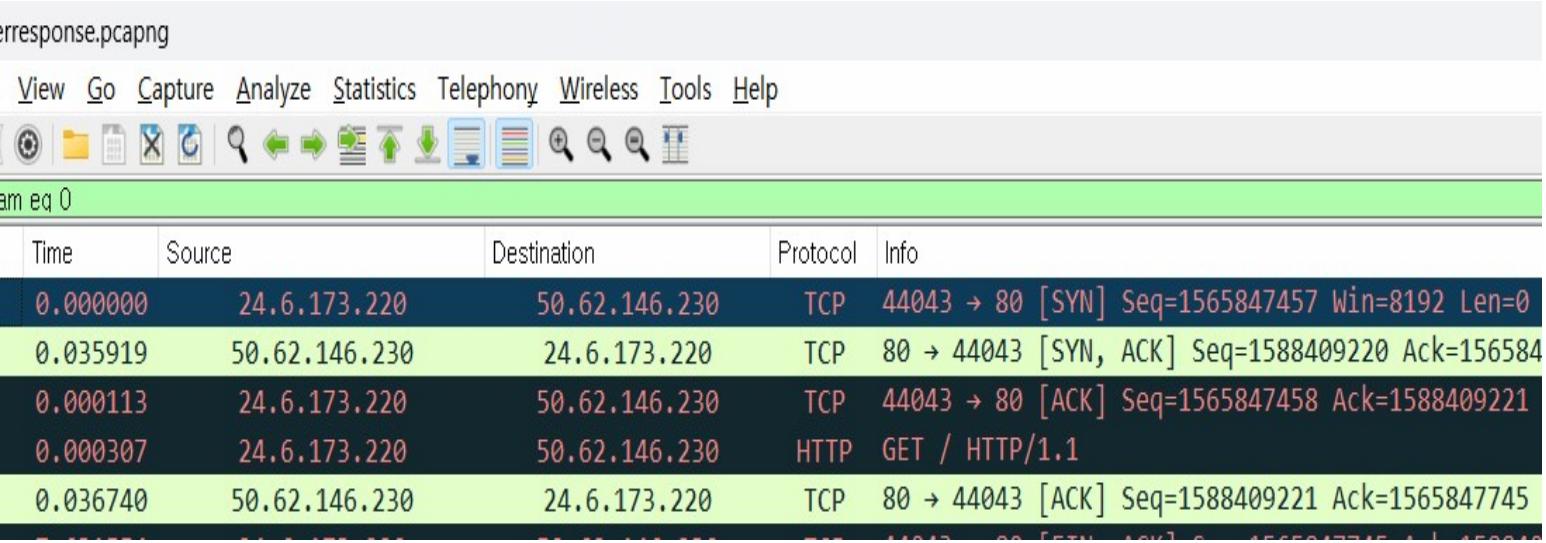
버 응답 부재

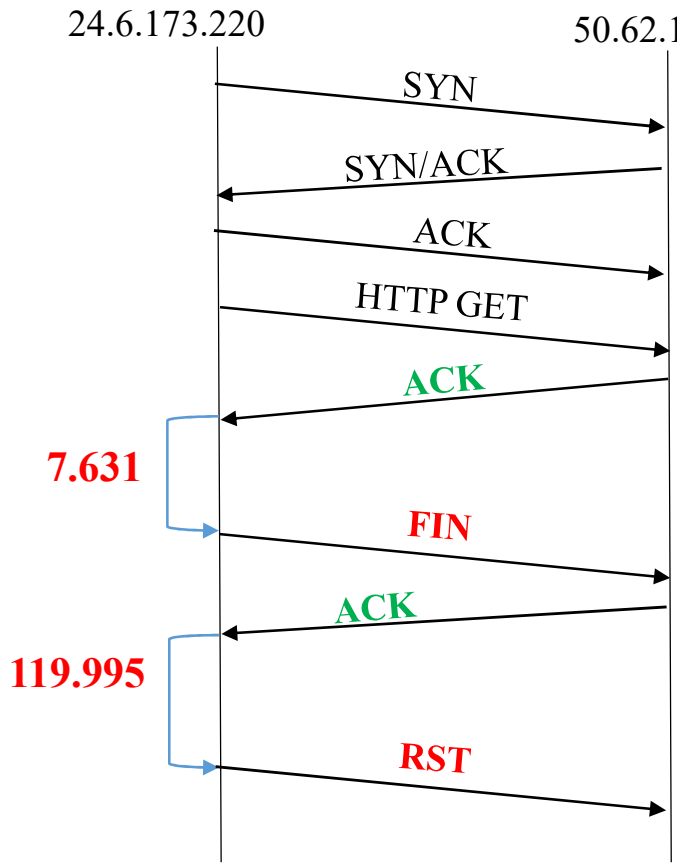

erver.pcapng
View Go Capture Analyze Statistics Telephony Wireless Tools Help
am eq 0
Time Source Destination Protocol Info
0.000000 192.168.1.72 192.168.1.66 TCP 5538 → 80 [SYN] Seq=1852461221 Win=8192


erver.pcapng
View Go Capture Analyze Statistics Telephony Wireless Tools Help
am eq 1
Time Source Destination Protocol Info
0.000000 192.168.1.72 192.168.1.66 TCP 5537 → 80 [SYN] Seq=3665406783 Win=8192


erver.pcapng
View Go Capture Analyze Statistics Telephony Wireless Tools Help
am eq 2
Time Source Destination Protocol Info
0.000000 192.168.1.72 192.168.1.66 TCP 5539 → 80 [SYN] Seq=3097499662 Win=8192

서비스 요청 무응답



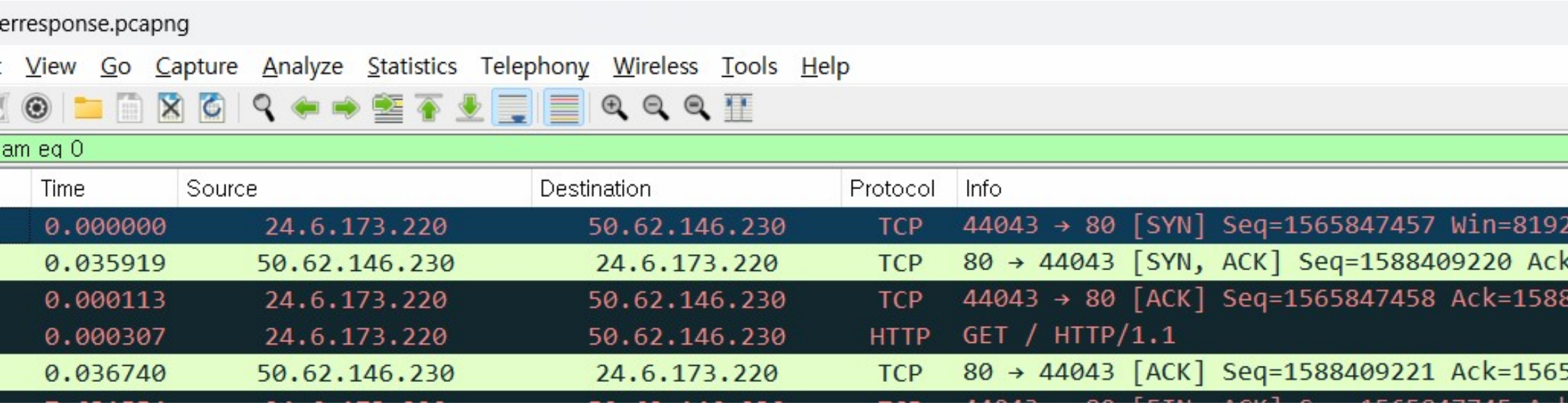


버의 HTTP 포트도 열려 있고, TCP도 정상적으로 운영이 되고 있으나 HTTP 데이터 송이 안되고 있음

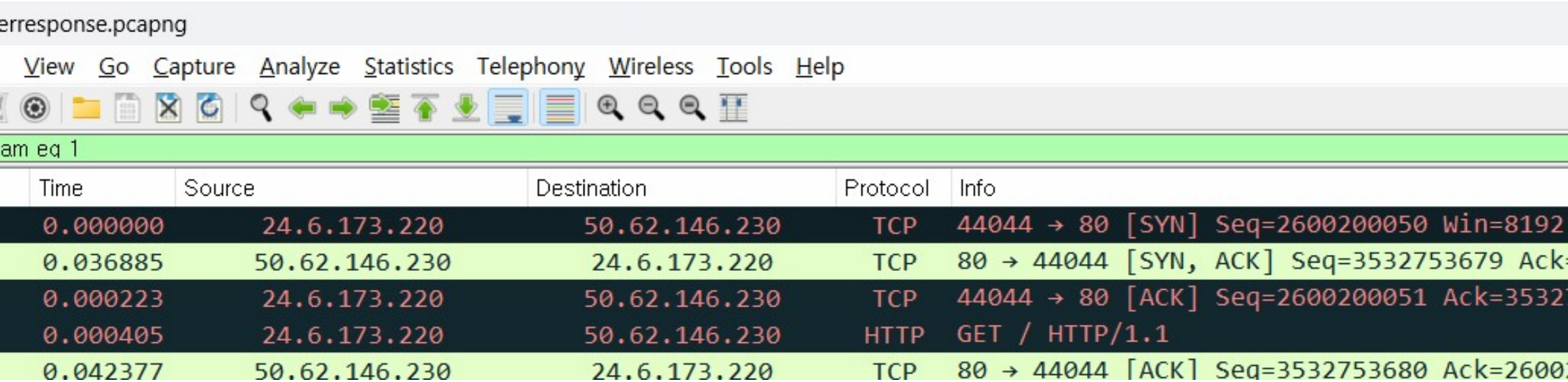
일한 패턴이 스트림별로 지속적어로 진행되면 네트워크 상의 결합이나 서버의 서비스 행의 일회성 문제가 아님

버 측 애플리케이션의 문제로 판단 가능 ➔ Application 문제로 판단 가능

비스 요청 무응답



서버의 HTTP 포트도 열려 있고, TCP도 정상적으로 운영이 되고 있으나 HTTP 데이터 전송이 안되고 있음
➔ Application 문제로 판단 가능



높은 대기시간(지연)의 원인 찾기 (P.327)

- 패킷 손실이 지연을 야기하지 않을 수 있음
- 두 호스트 간 통신이 느린 경우에도 TCP 재전송이나 중복 ACK 증상이 없을 수 있음
- 높은 지연의 근원지를 찾기 위한 가장 효과적인 방법
  - 초기의 3WHS와 이후 과정에서 최초의 패킷 쌍을 찾는 것
- **높은 지연 상황과 비교를 위해 정상적인 통신에 대한 기준이 필요**

❶ 정상적인 통신

View > Time Display Format > Second since Previous Displayed Packe

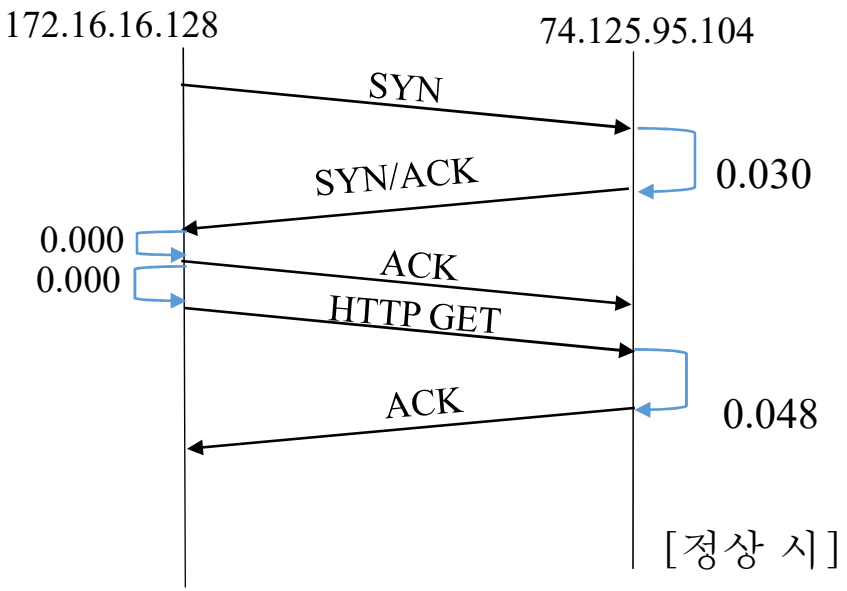
.pcapng

View Go Capture Analyze Statistics Telephony Wireless Tools Help

display filter ··· <Ctrl-/>

me	Source	Destination	Protocol	Info
.000000	172.16.16.128	74.125.95.104	TCP	1606 → 80 [SYN] Seq=2082691767 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM
.030107	74.125.95.104	172.16.16.128	TCP	80 → 1606 [SYN, ACK] Seq=2775577373 Ack=2082691768 Win=5720 Len=0 MSS=1406 SACK_PERM WS=64
.000075	172.16.16.128	74.125.95.104	TCP	1606 → 80 [ACK] Seq=2082691768 Ack=2775577374 Win=16872 Len=0
.000066	172.16.16.128	74.125.95.104	HTTP	GET / HTTP/1.1
.048778	74.125.95.104	172.16.16.128	TCP	80 → 1606 [ACK] Seq=2775577374 Ack=2082692395 Win=6976 Len=0
.022176	74.125.95.104	172.16.16.128	TCP	80 → 1606 [ACK] Seq=2775577374 Ack=2082692395 Win=6976 Len=1406 [TCP segment of a reassemble

체 프로세스가 **0.1**초가 채 걸리지 않을 정도로 속도가 빠름



속도가 느린 통신(호스트 간의 유선 지연)

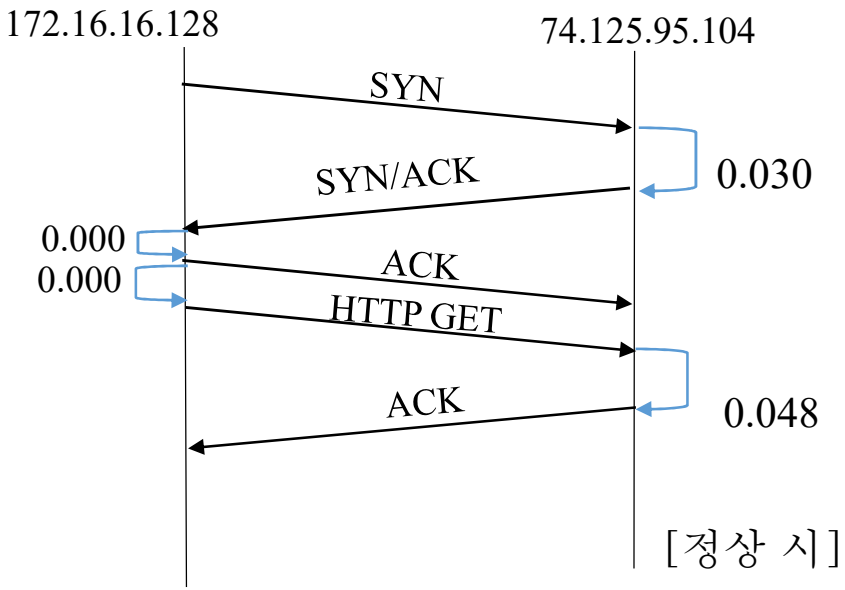
ncy2.pcapng

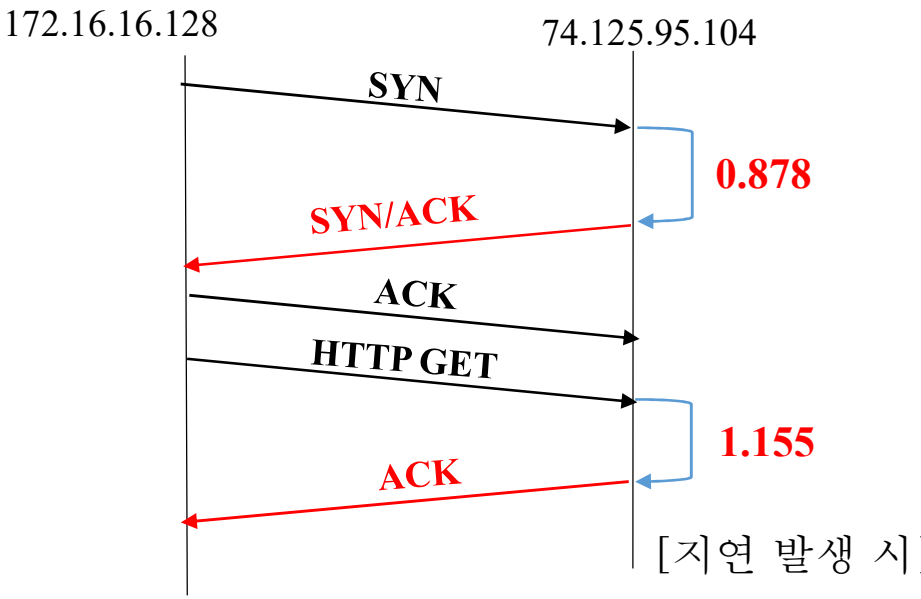
dit View Go Capture Analyze Statistics Telephony Wireless Tools Help

y a display filter ··· <Ctrl-/>

패킷 2와 5는 높은 지연 시간을 보여줌

Time	Source	Destination	Protocol	Info
0.000000	172.16.16.128	74.125.95.104	TCP	1606 → 80 [SYN] Seq=2082691767 Win=8192 Len=0 MSS=1460 WS=4 SACK_PERM
0.878530	74.125.95.104	172.16.16.128	TCP	80 → 1606 [SYN, ACK] Seq=2775577373 Ack=2082691768 Win=5720 Len=0 MSS=1406 SACK_PERM WS=64
0.016604	172.16.16.128	74.125.95.104	TCP	1606 → 80 [ACK] Seq=2082691768 Ack=2775577374 Win=16872 Len=0
0.000335	172.16.16.128	74.125.95.104	HTTP	GET / HTTP/1.1
1.155228	74.125.95.104	172.16.16.128	TCP	80 → 1606 [ACK] Seq=2775577374 Ack=2082692395 Win=6976 Len=0
0.015866	74.125.95.104	172.16.16.128	TCP	80 → 1606 [ACK] Seq=2775577374 Ack=2082692395 Win=6976 Len=1406 [TCP segment of a reassemb





❸ 속도가 느린 통신(클라이언트 지연)

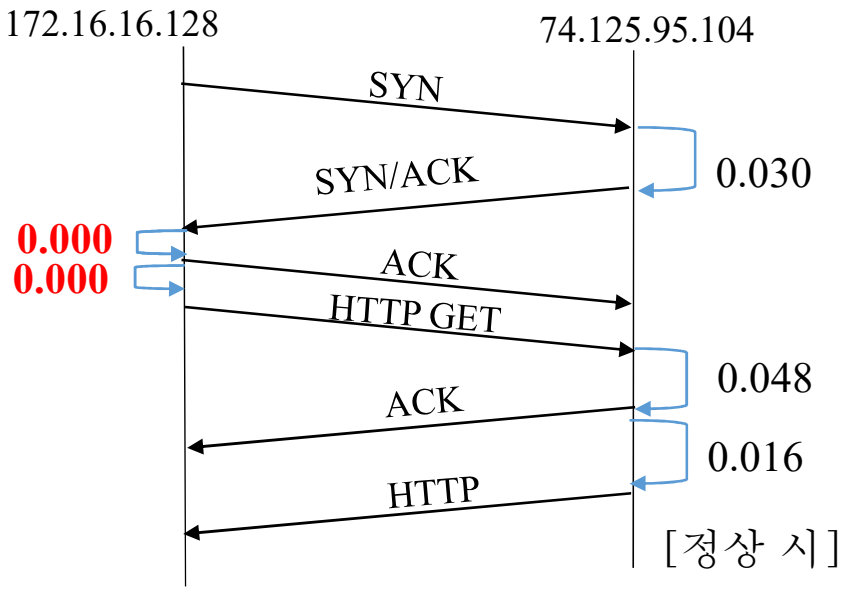
- ACK에서 GET으로 전환이 신속하게 이뤄지지 않음
- GET패킷의 생성과 전송은 응용계층까지의 처리가 필요

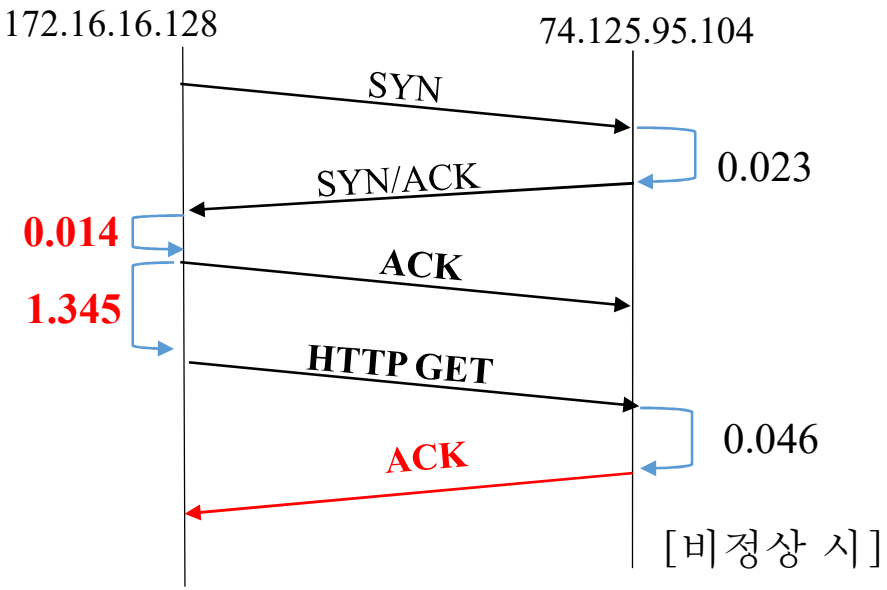
latency3.pcapng

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Apply a display filter ··· <Ctrl-/>

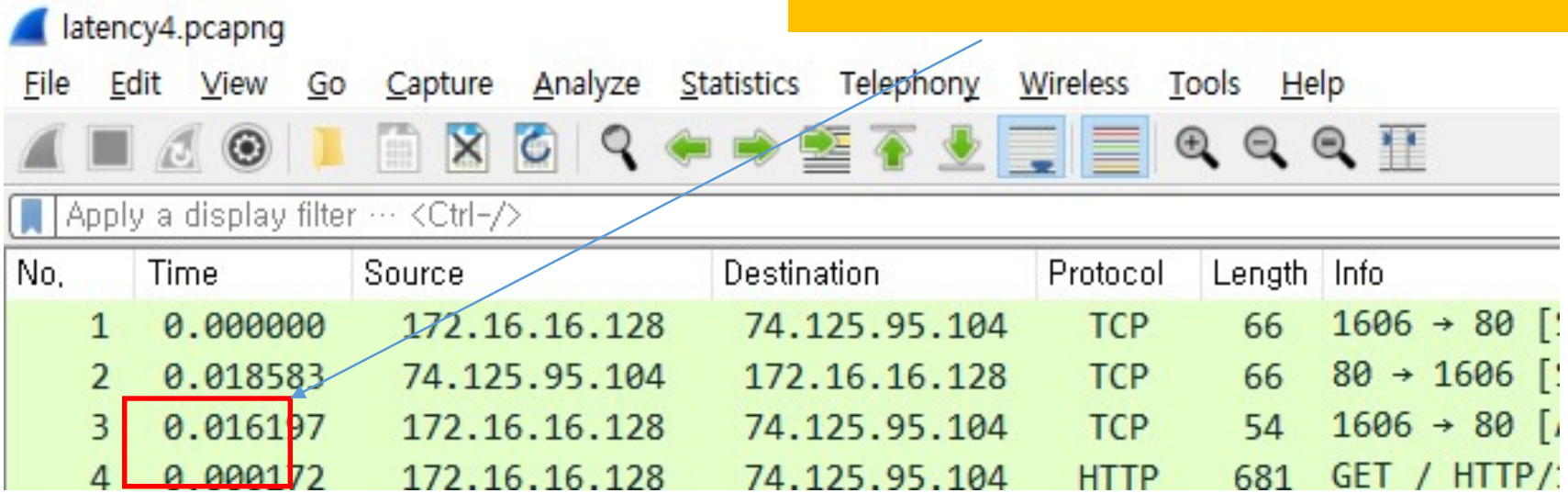
No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.16.16.128	74.125.95.104	TCP	66	1606 → 80
2	0.023790	74.125.95.104	172.16.16.128	TCP	66	80 → 1606
3	0.014894	172.16.16.128	74.125.95.104	TCP	54	1606 → 80
4	1.345023	172.16.16.128	74.125.95.104	HTTP	681	GET / HTT



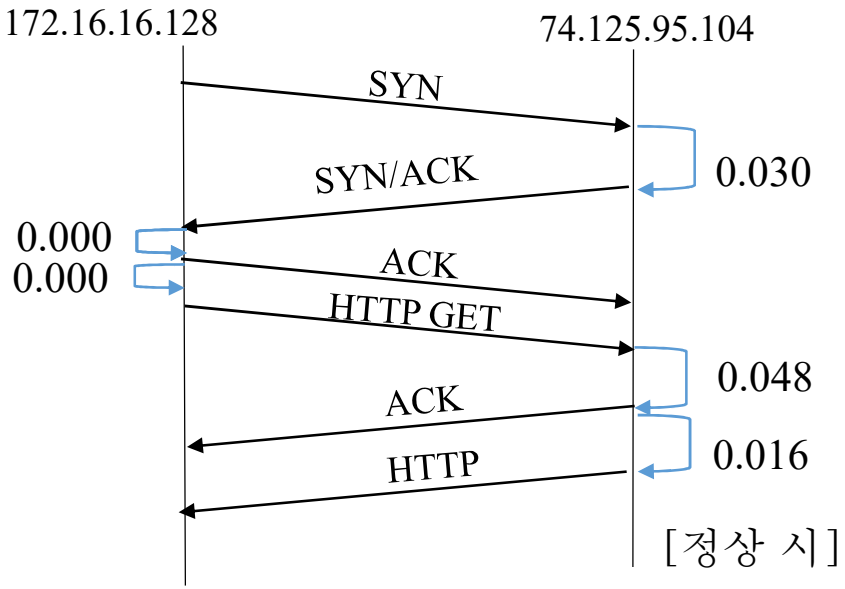


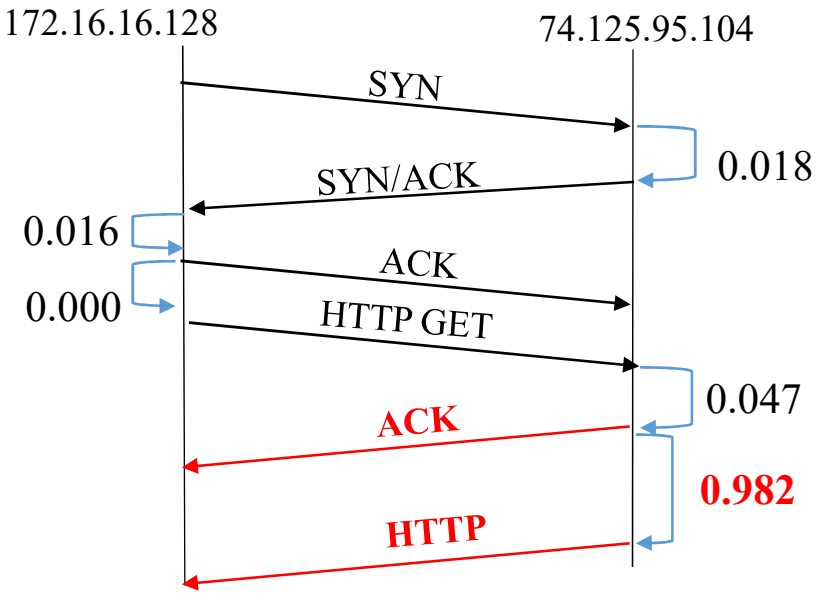
❹ 속도가 느린 통신(서버 지연)

- **ACK를 보내고 바로 서버는 데이터를 보내야 함**
- **서버가 적절한 시간 내에 데이터를 처리하지 못했음을 나타냄**

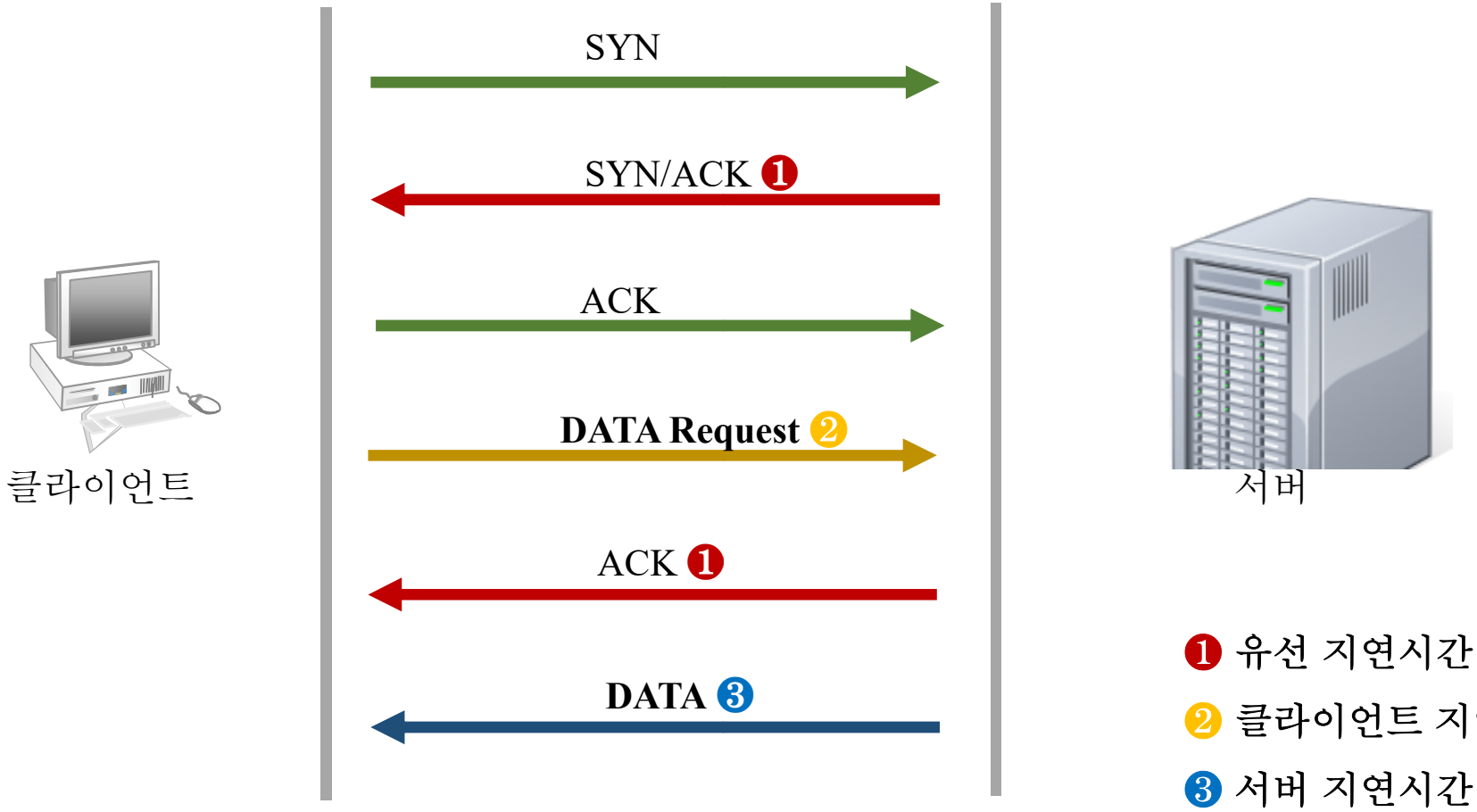


No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	172.16.16.128	74.125.95.104	TCP	66	1606 → 80 [
2	0.018583	74.125.95.104	172.16.16.128	TCP	66	80 → 1606 [
3	0.016197	172.16.16.128	74.125.95.104	TCP	54	1606 → 80 [
4	0.000172	172.16.16.128	74.125.95.104	HTTP	681	GET / HTTP/





높은 지연의 근원지 탐색



❶ 유선 지연시간

❷ 클라이언트 지연시간

❸ 서버 지연시간

TCP 오류 제어(P.295)

- 재전송 타이머
- 중복 ACK와 빠른 재전송

*재전송 타이머와 중복 ACK는 <u>패킷 손실을 복구하기 위한 TCP 기능</u>

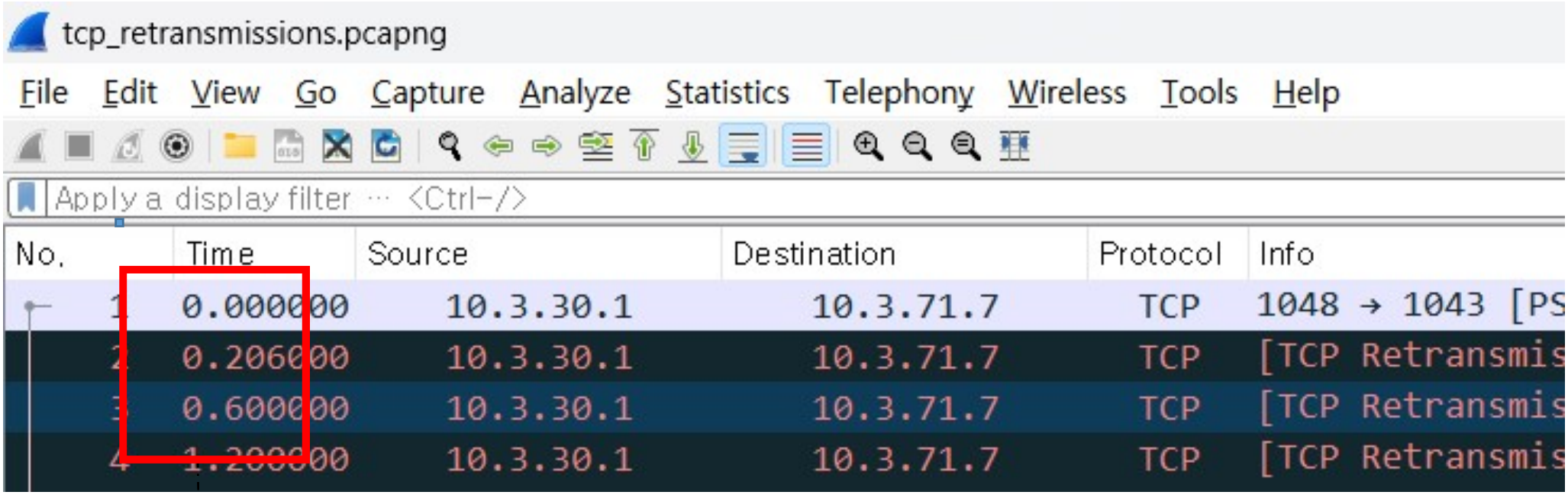
- **패킷 손실 원인**
  - 애플리케이션 오작동
  - 라우터 과부하
  - 일시적인 서비스 정지 등
- 패킷 손실은 일시적이기 때문에 패킷 손실을 탐지하고 복구가 중요
- **패킷 재전송 여부를 결정하는 주요 매커니즘 : 재전송 타이머(retransmission timer)**
  - 재전송 때마다 재전송 타이머가 동작하고 패킷을 정상을 받았다는 ACK 패킷을 받으면 타이머는 정
  - RTT(Round Trip Time) : 패킷 전송 시간과 해당 패킷의 ACK 수신 시간
  - RTO(Retransmission TimeOut) : RTT들의 평균

재전송 타이머를 이용한 패킷 손실 확인

- RTO(Retransmission TimeOut)값이 결정되기 전까지 RTT 디폴트 값 사용
- 수신된 패킷이 RTT를 통해 RTT 값이 조정되고 이를 기반으로 실제 RTO 값을 결정
- RTO 값이 결정되면 패킷 손실이 발생했는지 확인 가능

TCP Retransmission Timer

View > Time Display Format > Second since Previous Captured Packet



2*0.1=0.206
2*0.3=0.6
2*0.6=1.2
2*1.2=2.4
2*2.4=4.8

* RTO 값이 재전송 후에 두 배가 되기 때문에 시간이 기하급수적으로 증가

TCP Retransmission 증상과 원인 (P.301)

[증상] Endpoint의 통신 속도가 많이 느려질 수 있음

[원인] L1 또는 L2계층상에서의 오류로 인해 발생 가능

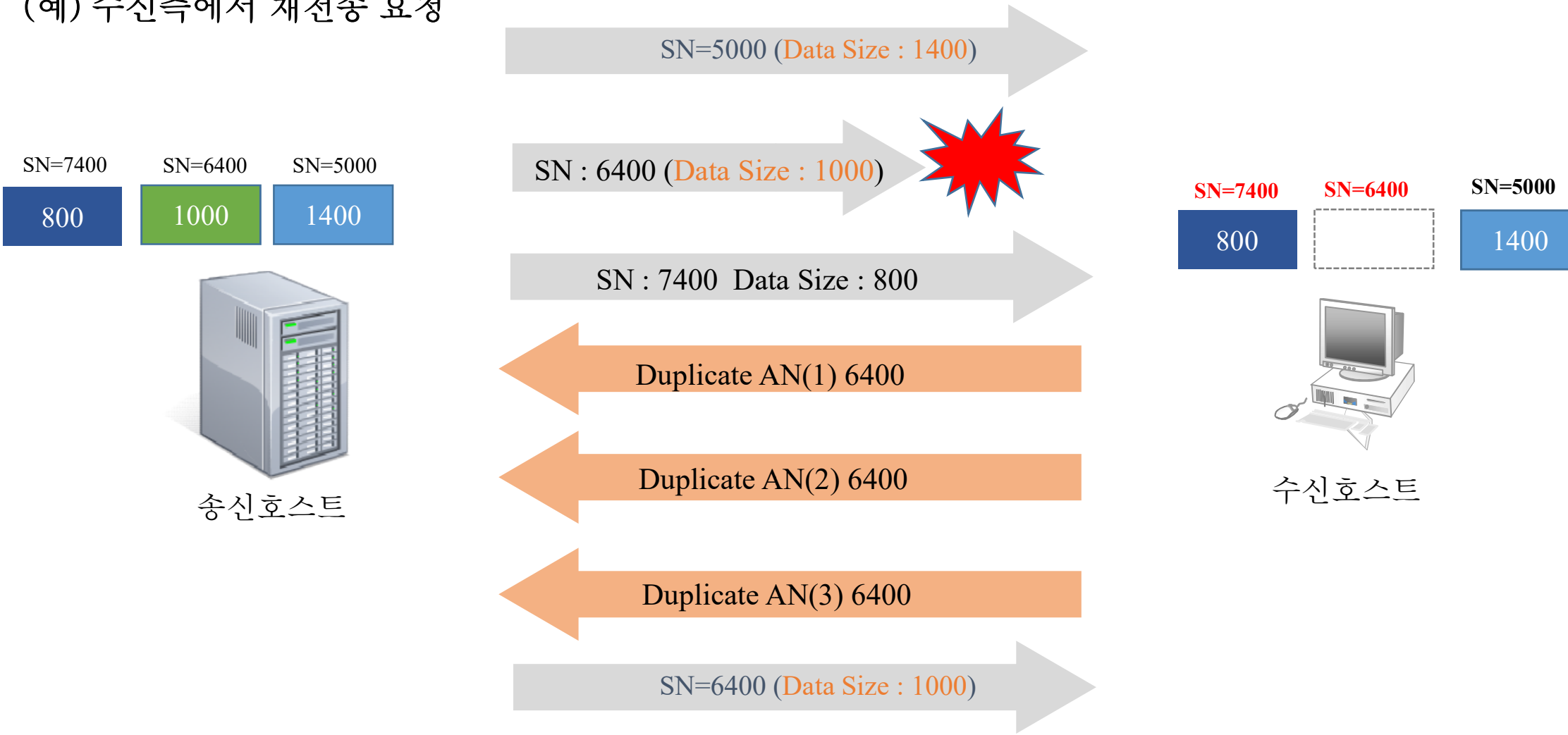
- Duplex mismatch로 인해 다수의 충돌 에러
- CRC 오류가 증가해 지속적인 재전송이 발생

TCP 중복 ACK와 빠른 재전송 (P.302)

- 패킷 손실을 복구하기 위해 고안된 TCP 기능
- 수신자가 잘못된 패킷을 받았을 때 보내는 TCP 패킷
- 중복 ACK는 패킷이 잘못된 순서로 수신될 때 발생

TCP 중복 ACK과 빠른 재전송

(예) 수신측에서 재전송 요청

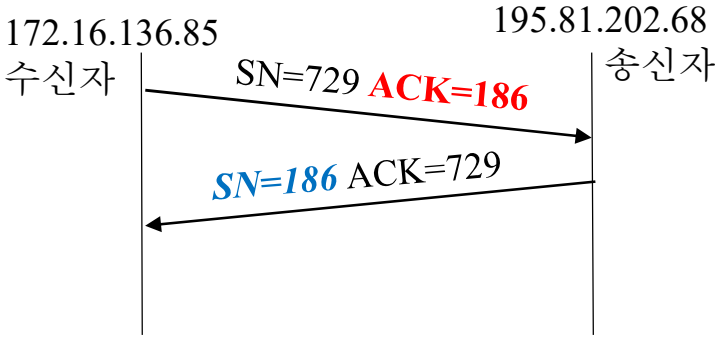


dupack.pcapng

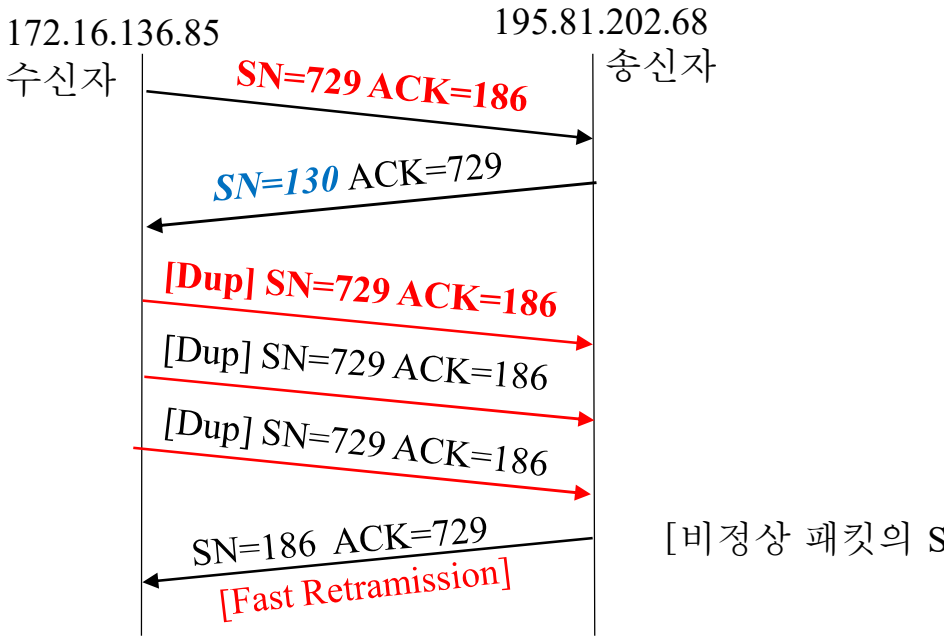
Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

y a display filter ··· <Ctrl-/>

Time	Source	Destination	Protocol	Info
0.000000	172.31.136.85	195.81.202.68	TCP	38760 → 80 [ACK] Seq=704338729 Ack=1310973186 Win=382 Len=0
0.000190	195.81.202.68	172.31.136.85	TCP	80 → 38760 [ACK] Seq=1310984130 Ack=704338729 Win=108 Len=136
0.000011	172.31.136.85	195.81.202.68	TCP	[TCP Dup ACK 1#1] 38760 → 80 [ACK] Seq=704338729 Ack=1310973
0.000093	195.81.202.68	172.31.136.85	TCP	80 → 38760 [ACK] Seq=1310985498 Ack=704338729 Win=108 Len=136
0.000010	172.31.136.85	195.81.202.68	TCP	[TCP Dup ACK 1#2] 38760 → 80 [ACK] Seq=704338729 Ack=1310973
0.000121	195.81.202.68	172.31.136.85	TCP	80 → 38760 [ACK] Seq=1310986866 Ack=704338729 Win=108 Len=136



[정상 패킷의 SN]



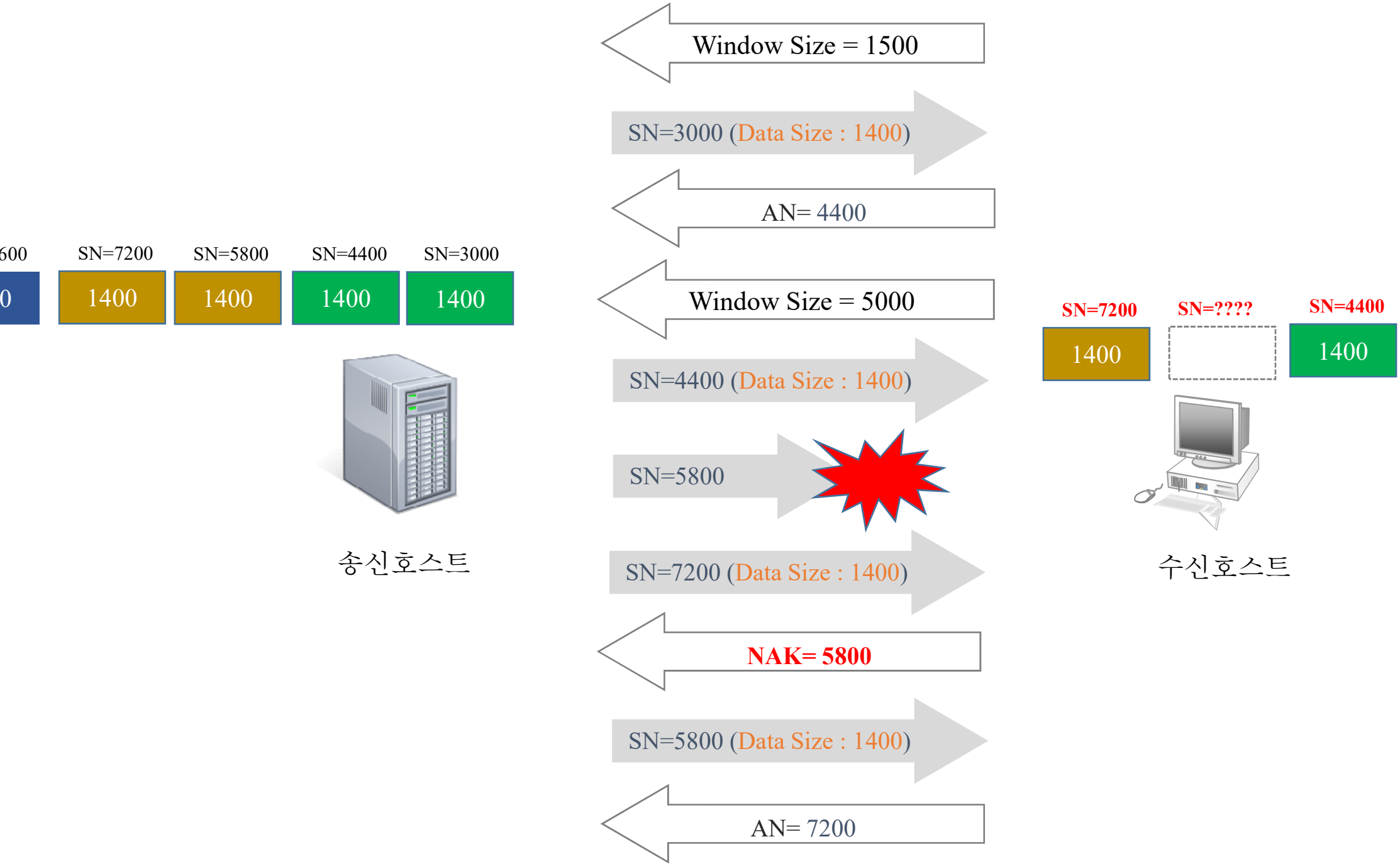
[비정상 패킷의 S

TCP Header – option

Selective Acknowledgment Option (SACK)

- TCP 는 out-of-sequence 데이터를 받았을 때 정상적으로 받은 마지막 패킷의 sequence number를 ack number로 송신측에 보내게 됨
  - 만약 홀(데이터 스트림중 중간부분에 손실)이 발생했을 경우 이미 전송된 데이터도 같이 재전송
  - 이는 TCP performance를 저하시킴
- 이를 보완하기 위해서 **selective acknowledge** 기능을 지원
  - 수신자는 수신한 세그먼트에서 홀을 찾아내서 selective ack를 보냄
  - **송신자는 selective ack를 수신하면 loss된 패킷들만 다시 재전송**
- selective ack TCP option은 SYN packets에서 이 옵션의 지원 여부를 교환
- 만약 peer가 이 옵션을 지원하지 않으면 SACK-permit option을 전송

kind = 4	length = 2


Window Size = 1500
SN=3000 (Data Size : 1400)
AN= 4400
Window Size = 5000
SN=4400 (Data Size : 1400)
SN=5800
SN=7200 (Data Size : 1400)
NAK= 5800
SN=5800 (Data Size : 1400)
AN= 7200
600
SN=7200
SN=5800
SN=4400
SN=3000
0
1400
1400
1400
1400
송신호스트
SN=7200
SN=????
SN=4400
1400
1400
수신호스트

TCP 흐름 제어(P.310)

- 송수신자 사이의 전송속도(양)를 제어(조절)
- Sliding window을 이용

- 패킷 손실을 방지하기 위한 사전 예방 메커니즘

* **송신자가 수신자의 buffer overflow가 발생되지 않도록 전송량을 제어하는 매카니즘**

- 패킷 손실이 발생할 때를 감지하고 데이터 전송속도를 조정해 손실 방지

- 수신자의 receive window를 활용해 데이터 흐름을 제어

* Receive window : 수신지의 TCP 버퍼에 저장한 데이터 양

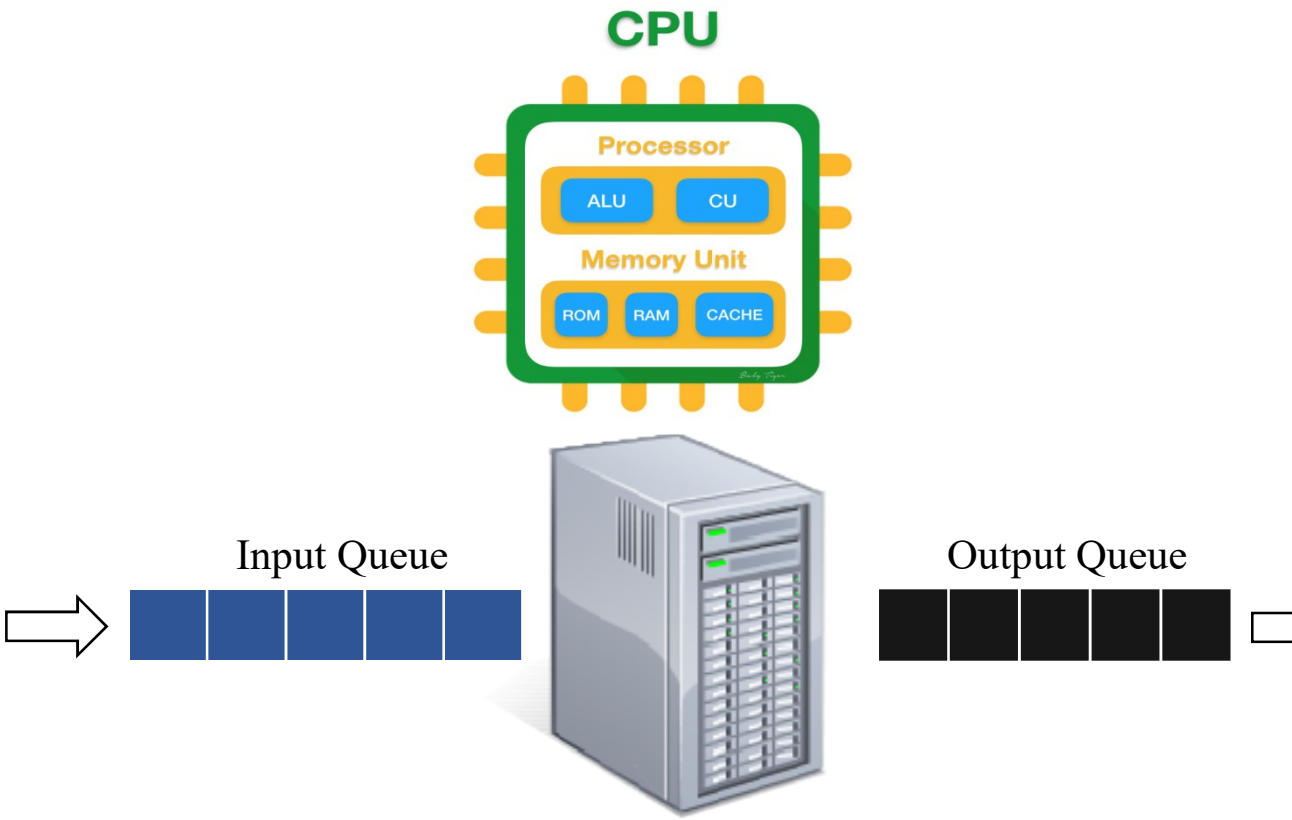
TCP receive window size에 변화를 줄 수 있는 3가지 요인

- TCP의 데이터 수신 속도(인터페이스)

- TCP 수신 버퍼 내의 데이터를 애플리케이션이 처리하는 속도(애플리케이션)

- TCP 수신 버퍼 자체가 적은 경우(물리적 메모리)

네트워크 지연 발생 원인

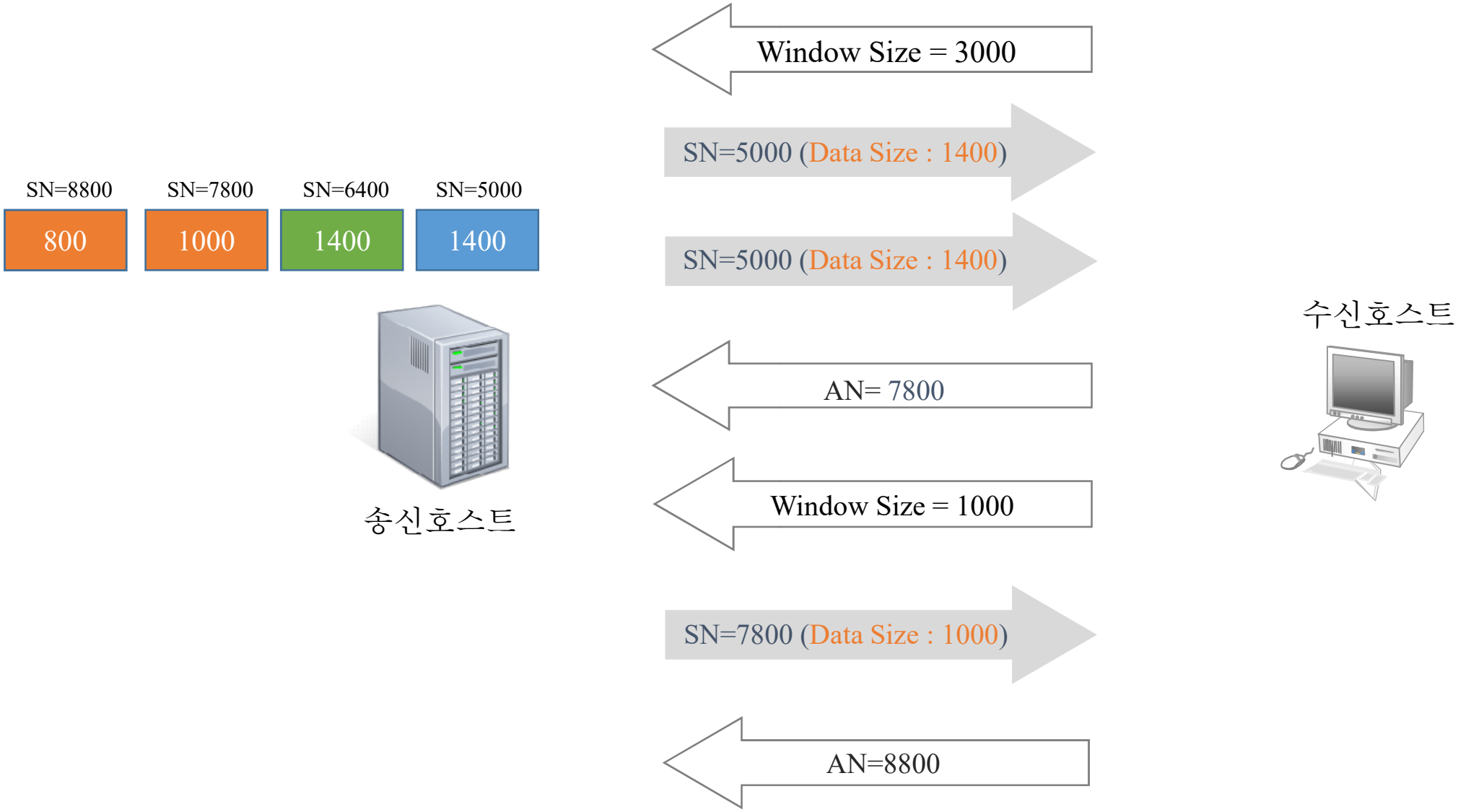


Output queue에서 drop 발생 원인

- 대역폭이 부족하면 output queue는 제때 비어지지 못하기 때문에 패킷들은 queue에 쌓이고, queue에 패킷이 차면 쌓인 패킷들은 폐기
- 인터페이스에 연결된 대역폭이 부족 시 대역폭 업그레이드가 필요

Input queue에서 drop 발생 원인

- CPU나 메모리 자원이 부족하면 input queue에서 제때 메모리로 옮겨지지 못하기 때문에 drop 발생
- 장비(메모리, CPU) 성능의 업그레이드가 필요

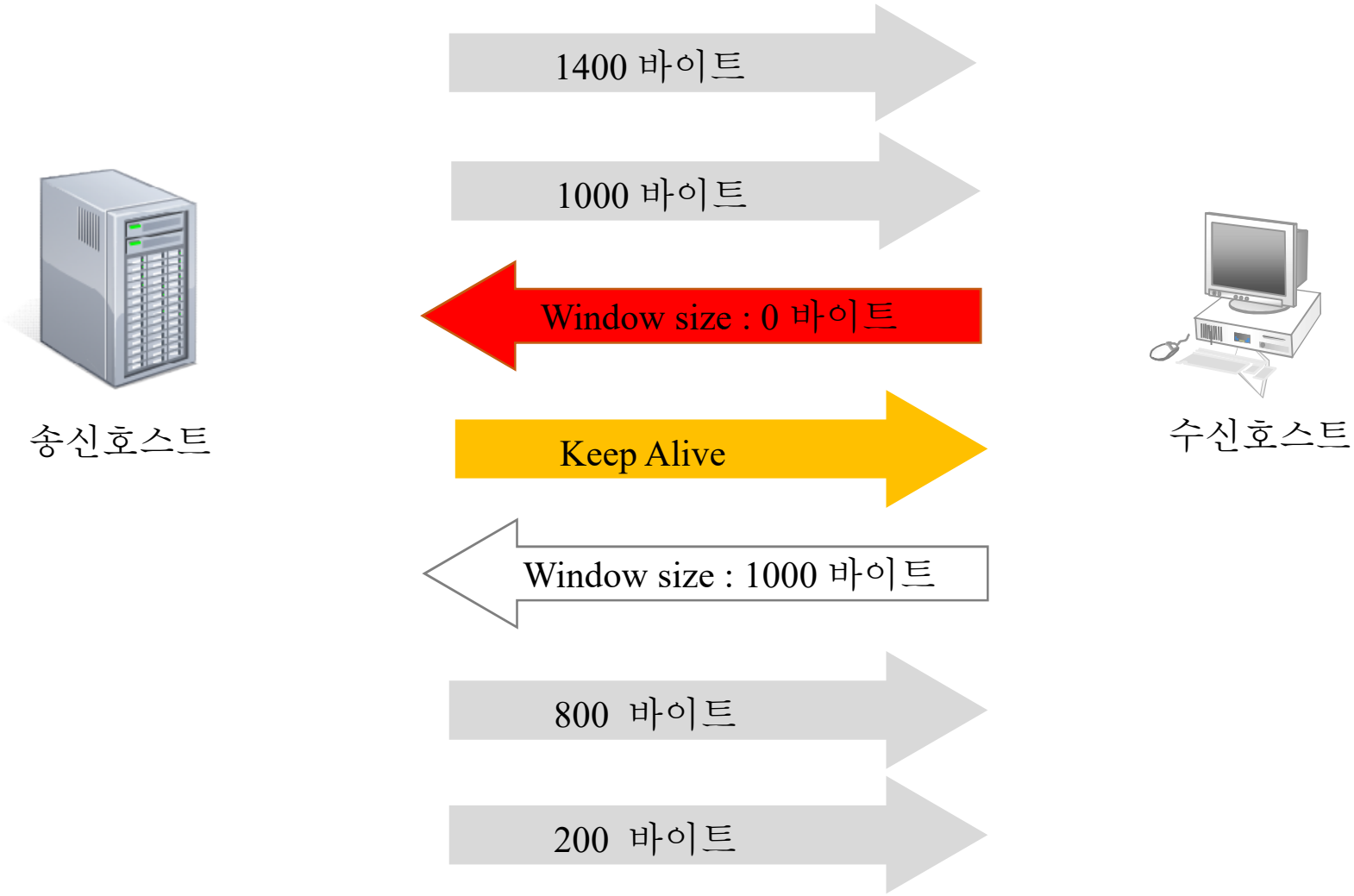

Window Size = 3000
SN=5000 (Data Size : 1400)
SN=8800
SN=7800
SN=6400
SN=5000
800
1000
1400
1400
SN=5000 (Data Size : 1400)
수신호스트
AN= 7800
Window Size = 1000
송신호스트
SN=7800 (Data Size : 1000)
AN=8800

Zero Window Size Packet (P.317)

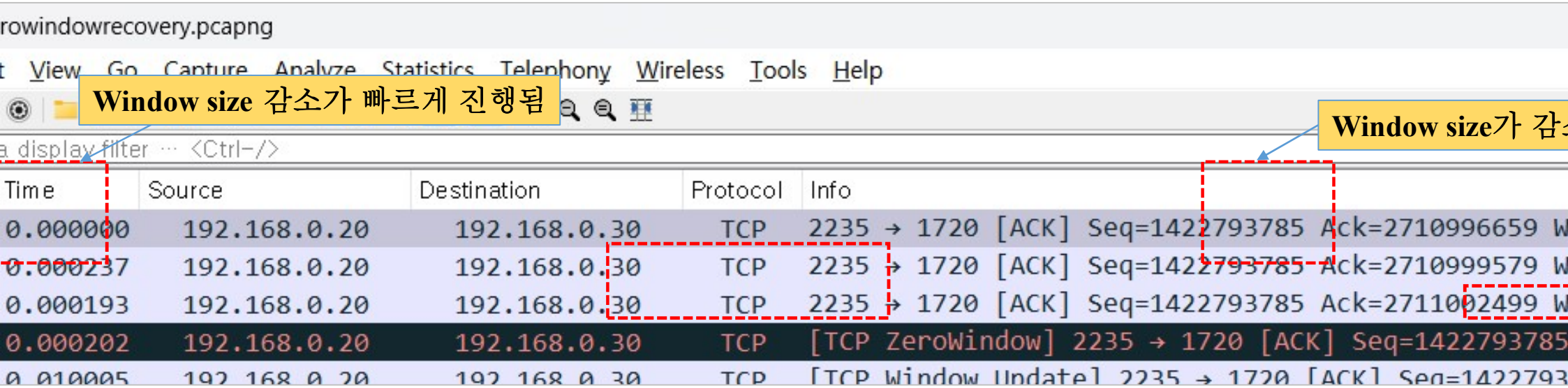
- 메모리 부족, 처리 능력 부족 또는 다른 문제점으로 인해 수신 데이터를 처리 못할 경우 보내는 패킷
- 송신자가 zero window size 패킷을 받으면 모든 데이터 전송을 중지
- 데이터 전송은 중지하지만 keep-alive 패킷 전송과 세션 연결은 유지

  * Keep-alive 패킷 : 정기적으로 송신지에서 수신지로 receive windows 상태를 확인하기 위해 보내는 데이터

- 수신지가 데이터 처리를 다시 시작할 수 있게 되면 0이 아닌 window size로 응답하고 통신을 재개


1400 바이트
1000 바이트
Window size : 0 바이트
송신호스트
Keep Alive
수신호스트
Window size : 1000 바이트
800 바이트
200 바이트

roWindow & Window Update

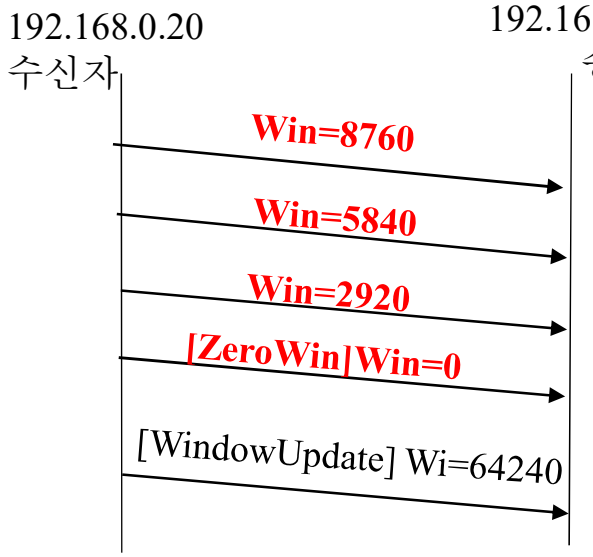


Window size 감소는 호스트로부터의 지연(delay)이 증가함을 알려줌

< Window update >>

애플리케이션이 수신버퍼에서 데이터를 read 할 경우 전송됨

- ZeroWindow가 발생하면 Window update가 유일한 해결방법



roWindow & Keep-Alive

cp_zerowindowdead.pcapng

Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

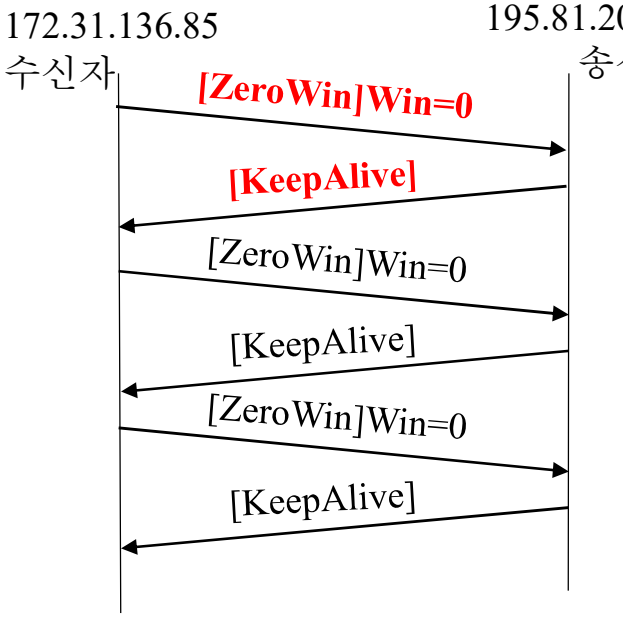
pply a display filter ··· <Ctrl-/>

	Time	Source	Destination	Protocol	Info
1	0.000000	195.81.202.68	172.31.136.85	TCP	80 → 38760 [PSH, ACK] Seq=1310996442 Ack=704
2	0.000029	172.31.136.85	195.81.202.68	TCP	[TCP ZeroWindow] 38760 → 80 [ACK] Seq=704338
3	3.410576	195.81.202.68	172.31.136.85	TCP	[TCP Keep-Alive] 80 → 38760 [ACK] Seq=131099
4	0.000031	172.31.136.85	195.81.202.68	TCP	[TCP ZeroWindow] 38760 → 80 [ACK] Seq=704338
5	6.784127	195.81.202.68	172.31.136.85	TCP	[TCP Keep-Alive] 80 → 38760 [ACK] Seq=131099
6	0.000029	172.31.136.85	195.81.202.68	TCP	[TCP ZeroWindow] 38760 → 80 [ACK] Seq=704338

eepAlive(연결유지패킷)은 3.4, 6.8, 13.5초 간격으로 발생

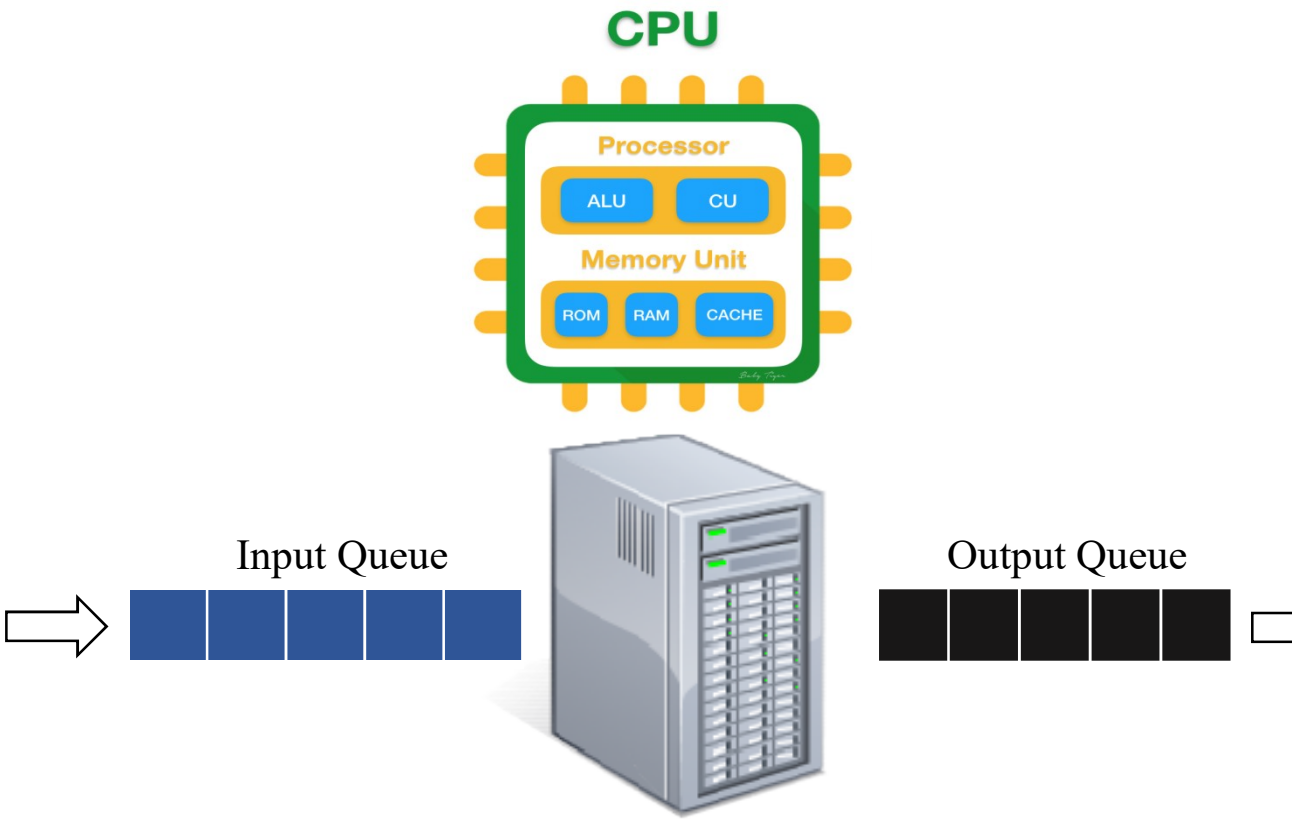
eroWindow 발생 원인은 수신 버퍼가 가득 찬 경우
수신 버퍼에서 데이터를 read하지 않은 애플리케이션 때문

< 버퍼에서 데이터를 애플리케이션이 read하지 않는 이유>>
애플리케이션의 처리 시간이 부족할 경우
애플리케이션의 잘못 만들어 졌을 경우
전송 지연을 유발하기 위해 고의로 zerowindow 패킷을 발생 할 경우



Sliding window

네트워크 지연 발생 원인



Output queue에서 drop 발생 원인

- 대역폭이 부족하면 output queue는 제때 비어지지 못하기 때문에 패킷들은 queue에 쌓이고, queue에 패킷이 차면 쌓인 패킷들은 폐기
- 인터페이스에 연결된 대역폭이 부족 시 대역폭 업그레이드가 필요

Input queue에서 drop 발생 원인

- CPU나 메모리 자원이 부족하면 input queue에서 제때 메모리로 옮겨지지 못하기 때문에 drop 발생
- 장비(메모리, CPU) 성능의 업그레이드가 필요

네트워크 지연 발생 원인

- 폐기된 패킷은 TCP 재전송 메커니즘에 의해 재전송되는데, 이 과정에서 전송속도가 길어 짐
  - 대역폭 부족이나 장비의 성능 부족은 패킷 폐기를 발생
  - 패킷 폐기는 패킷 지연을 발생시킴
- 패킷 지연은 응답시간을 길게 만들어 사용자의 불만 초래

⇨ **Congestion Avoidance**(혼잡 회피) 메커니즘이 필요

- 혼잡 제어를 위한 4가지 알고리즘

  - Slow start(느린시작)
  - Congestion Avoidance(혼잡회피)

  패킷 손실 전

  - Fast Retransmission(빠른 재전송)
  - Fast Recovery(빠른 회복)

  패킷 손실 후

Window size

- ACK 응답 없이 보낼 수 있는 패킷의 수
- Window size 크기는 송신장치가 계산한 CWND와 수신 장치가 보낸 중에서 작은 값으로 선정

❶ CWND (Congestion Window)

- 송신자의 네트워크 상태에 관한 값

❷ RWND(Received Window or Advertised Window)

- 수신 장비의 상태에 값으로 수신 장치의 windows size

큰 RWND를 보내도 서서히 증가하는 CWND의 특지 때문에 보내재는 데이터 양이 느리게 증가한

Slow start : CWND 크기 증가 기술

Slow Start

- TCP의 정상적인 상태에서의 동작 구현 알고리즘
  - CWND 크기의 느린 증가를 ‘slow start’라고 함
  - CWND 크기를 서서히 증가 시키는 것
  - CWND 를 서서히 증가시키는 이유는 망이 전송 능력을 파악하기 위함

- Slow start가 혼잡(congestion) 원인이 될 수 있음
  - 송수신 장치가 속도가 빠른 LAN으로 연결했다면 문제가 없음
  - 느린 라우터와 같은 WAN으로 연결될 경우 혼잡 발생

Slow Start

- TCP 연결에 사용할 수 있는 전송 속도를 탐지하기 위한 것
  - TCP는 물리적 링크의 데이터 전송속도를 알지 못함
  - TCP가 한꺼번에 많은 데이터를 전송할 경우 네트워크 장애 발생
- 적은 수의 세그먼트를 전송하다가 천천히 세그먼트 수를 증가
- Slow start의 두 가지 파라미터
  - CWND(Congestion Window, 혼잡 윈도우)
  - SSTHRESH(Slow-Start Thresahhold Value, 느린 시작임계값)

Slow Start

<< Congestion 확인 방법 2가지 >>

❶ 재전송 타이머를 보고 판단

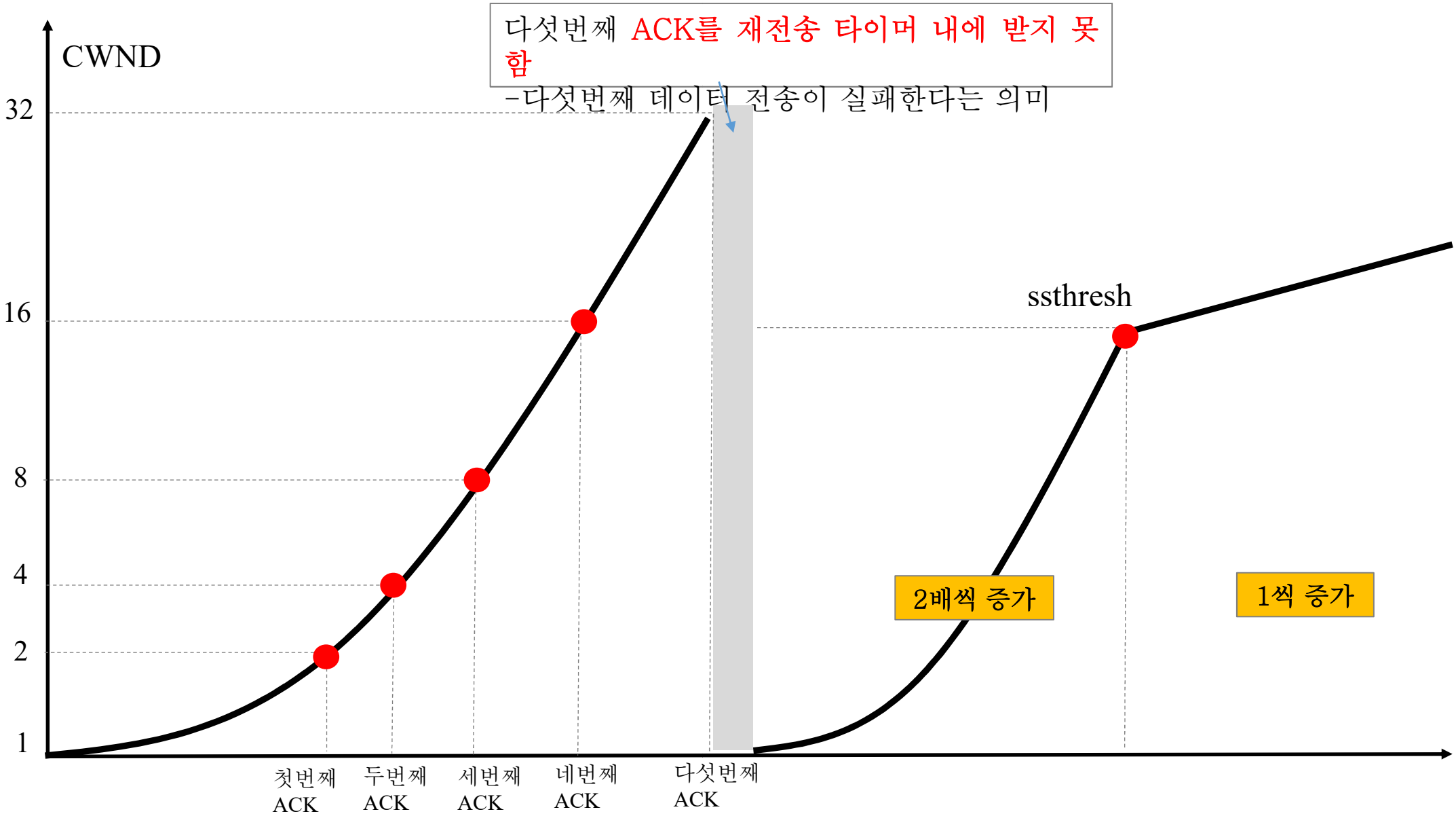
- 재전송 타이머 내에 ACK를 받지 못한 경우 CWND는 1에서 다시 시작
- sshthresh 값은 전송실패(재전송 발생) 전의 반 (sshthresh=CWND/2)
- sshthresh가 될 때까지 2배식 증가, congestion 이후부터는 1씩 증가

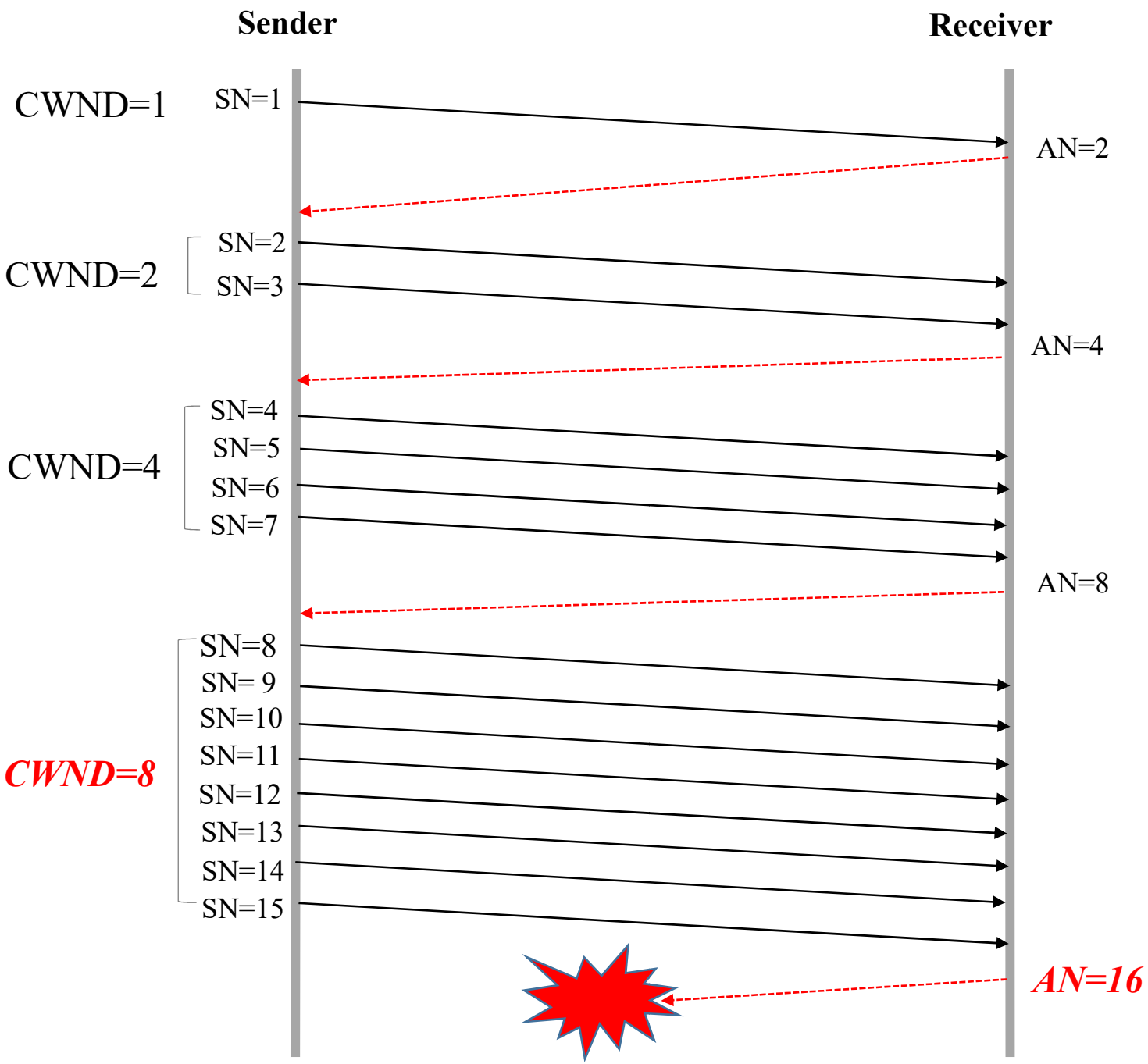
❷ 3개의 동일한 ACK가 연속적으로 들어오는 것을 보고 판단

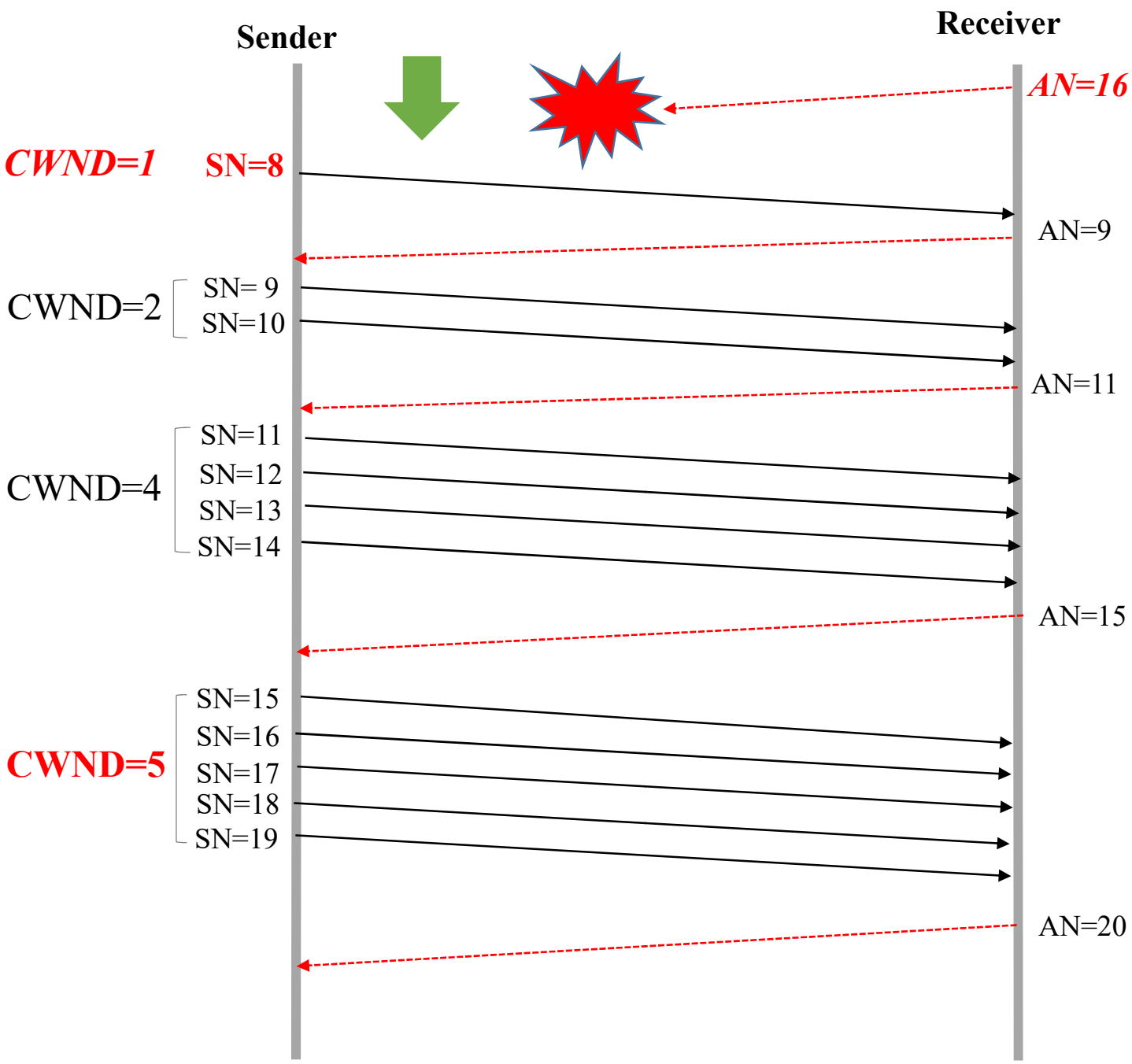
* 한 패킷의 유실은 congestion이 심각하지 않음 문제로 판단

- 빠른 복귀(fast recovery) : CWND의 크기를 1로 줄이지 않고 반(0.5)으로 줄임
- 빠른 재전송(fast retransmission) : 3개의 동일한 ACK를 받은 경우 타임아웃만큼 기다리지 않고 즉시 송

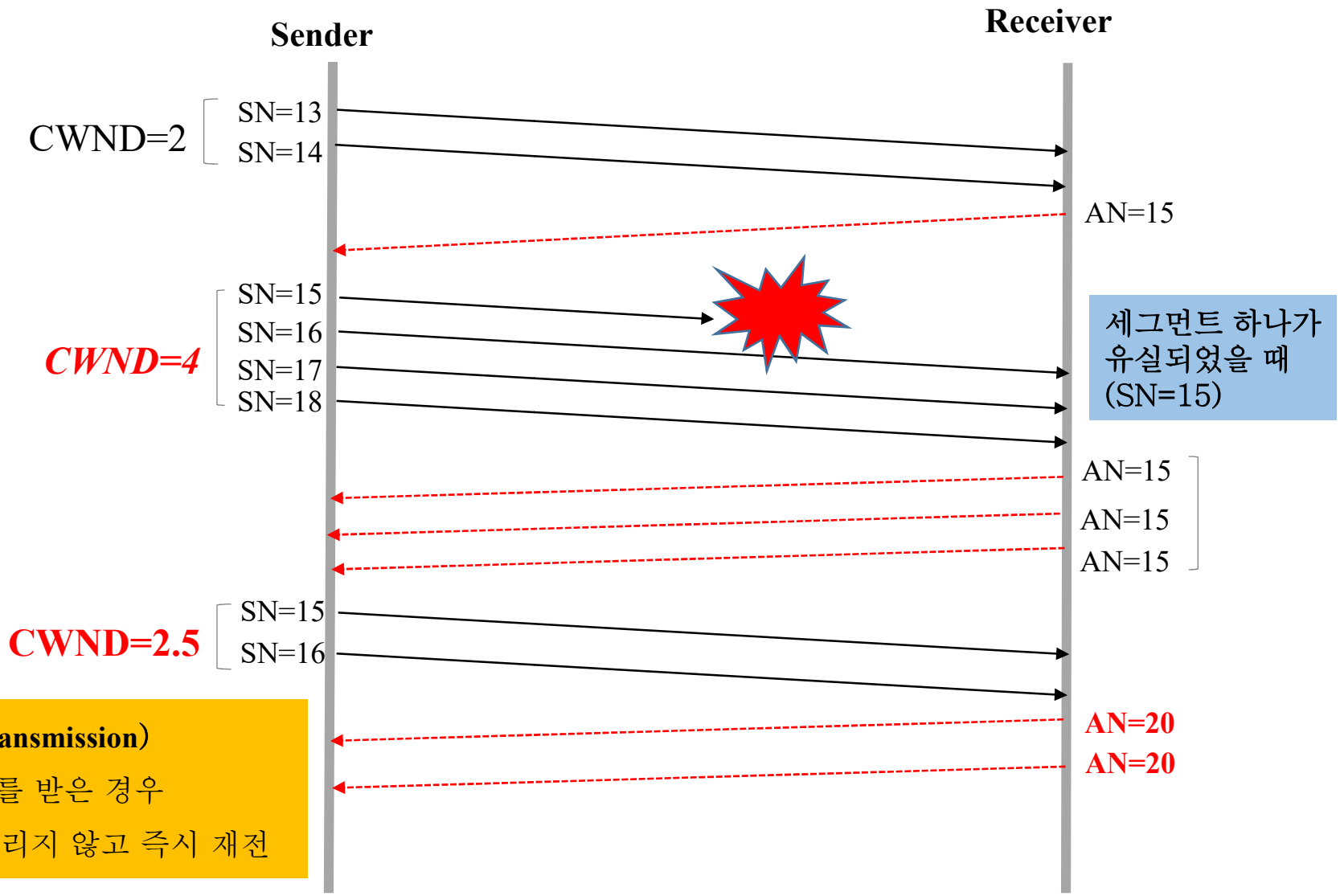
❶ 재전송 타이머를 보고 판단




Sender
Receiver
CWND=1
SN=1
AN=2
CWND=2
SN=2
SN=3
AN=4
CWND=4
SN=4
SN=5
SN=6
SN=7
AN=8
CWND=8
SN=8
SN= 9
SN=10
SN=11
SN=12
SN=13
SN=14
SN=15
AN=16


Sender
Receiver
AN=16
CWND=1
SN=8
AN=9
CWND=2
SN= 9
SN=10
AN=11
CWND=4
SN=11
SN=12
SN=13
SN=14
AN=15
CWND=5
SN=15
SN=16
SN=17
SN=18
SN=19
AN=20

❷ 3개의 동일한 ACK가 연속적으로 들어오는 것을 보고 판단

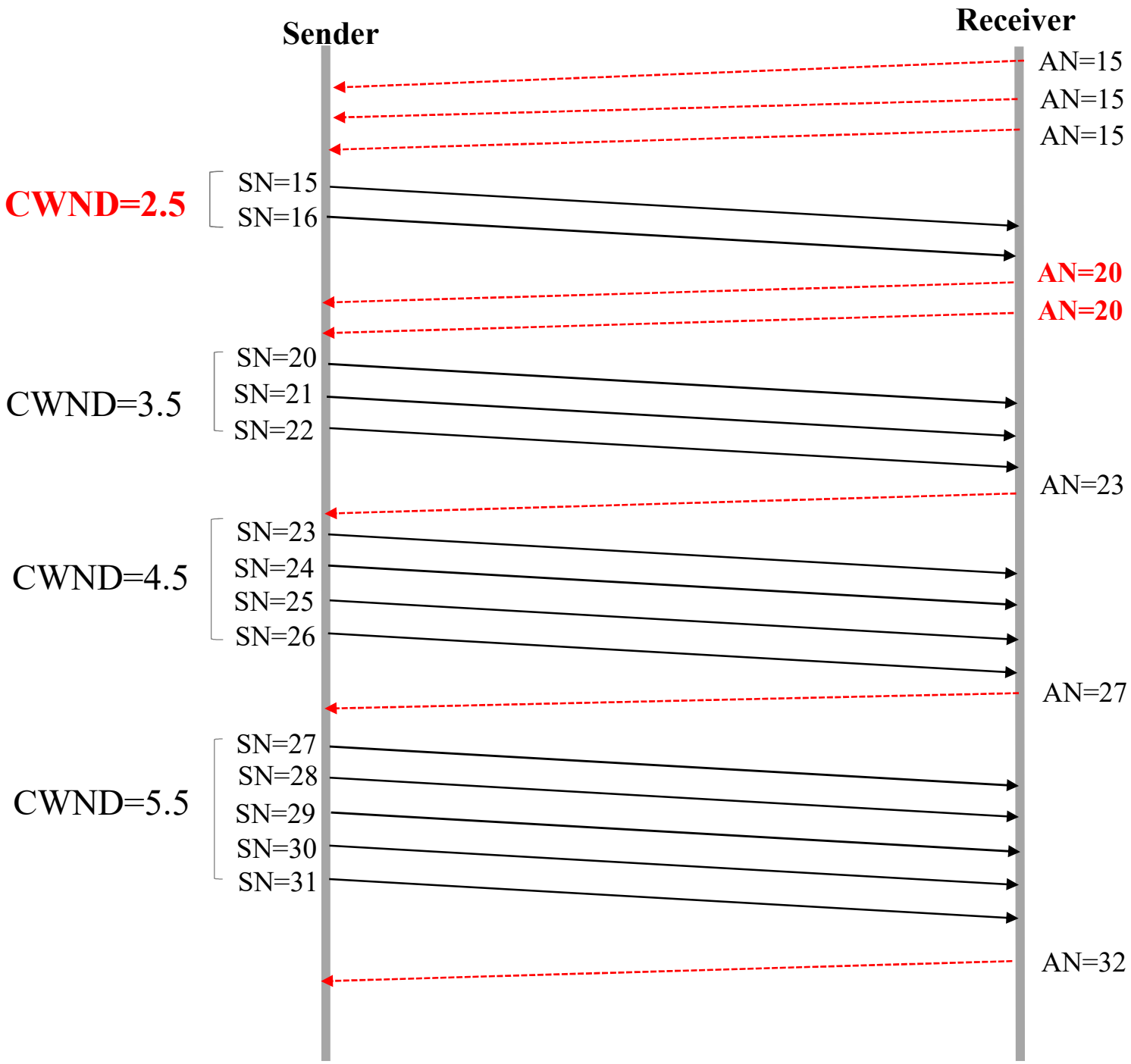


* **빠른 재전송(fast retransmission)**

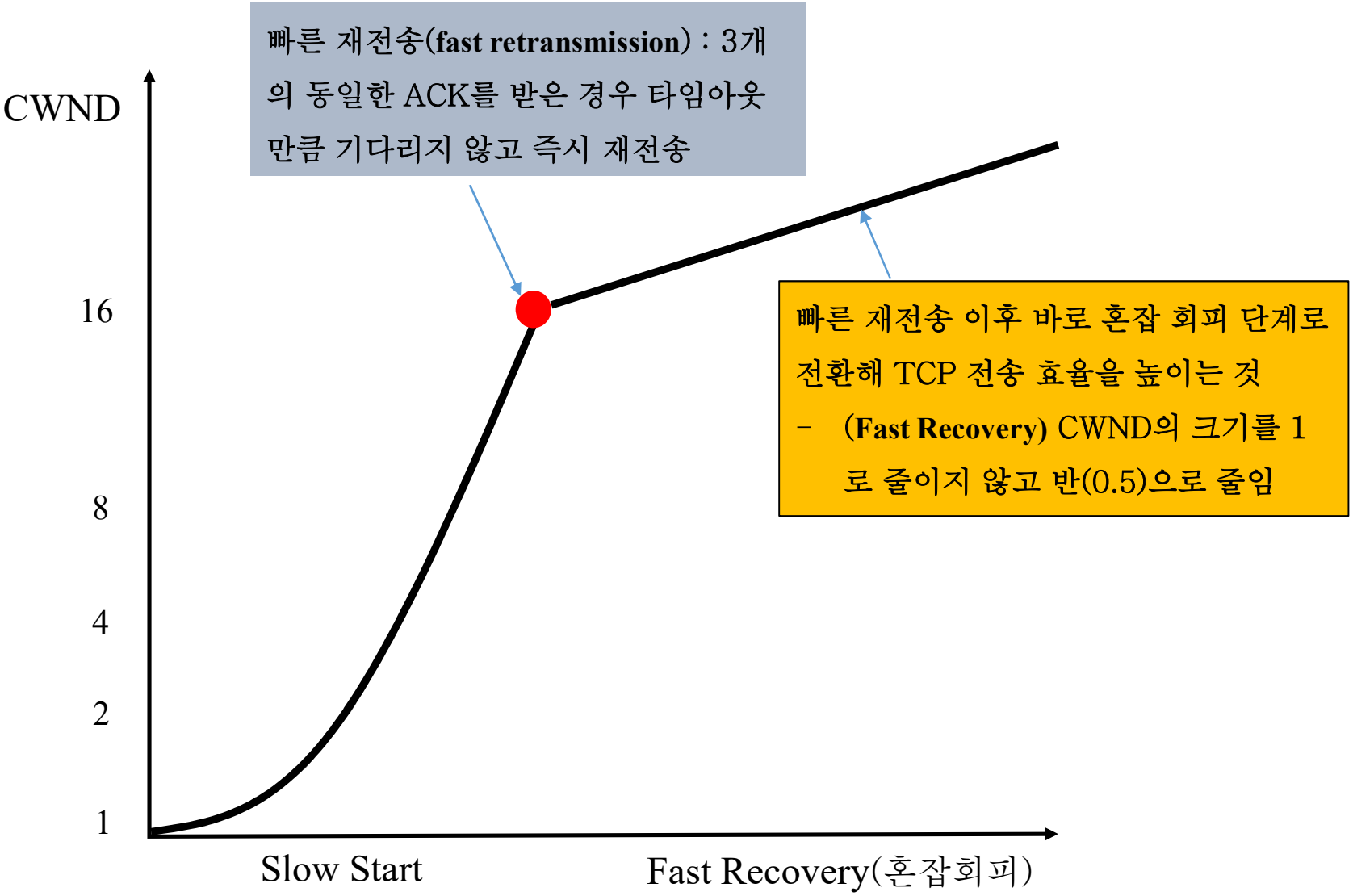
: 3개의 동일한 ACK를 받은 경우 타임아웃만큼 기다리지 않고 즉시 재전송

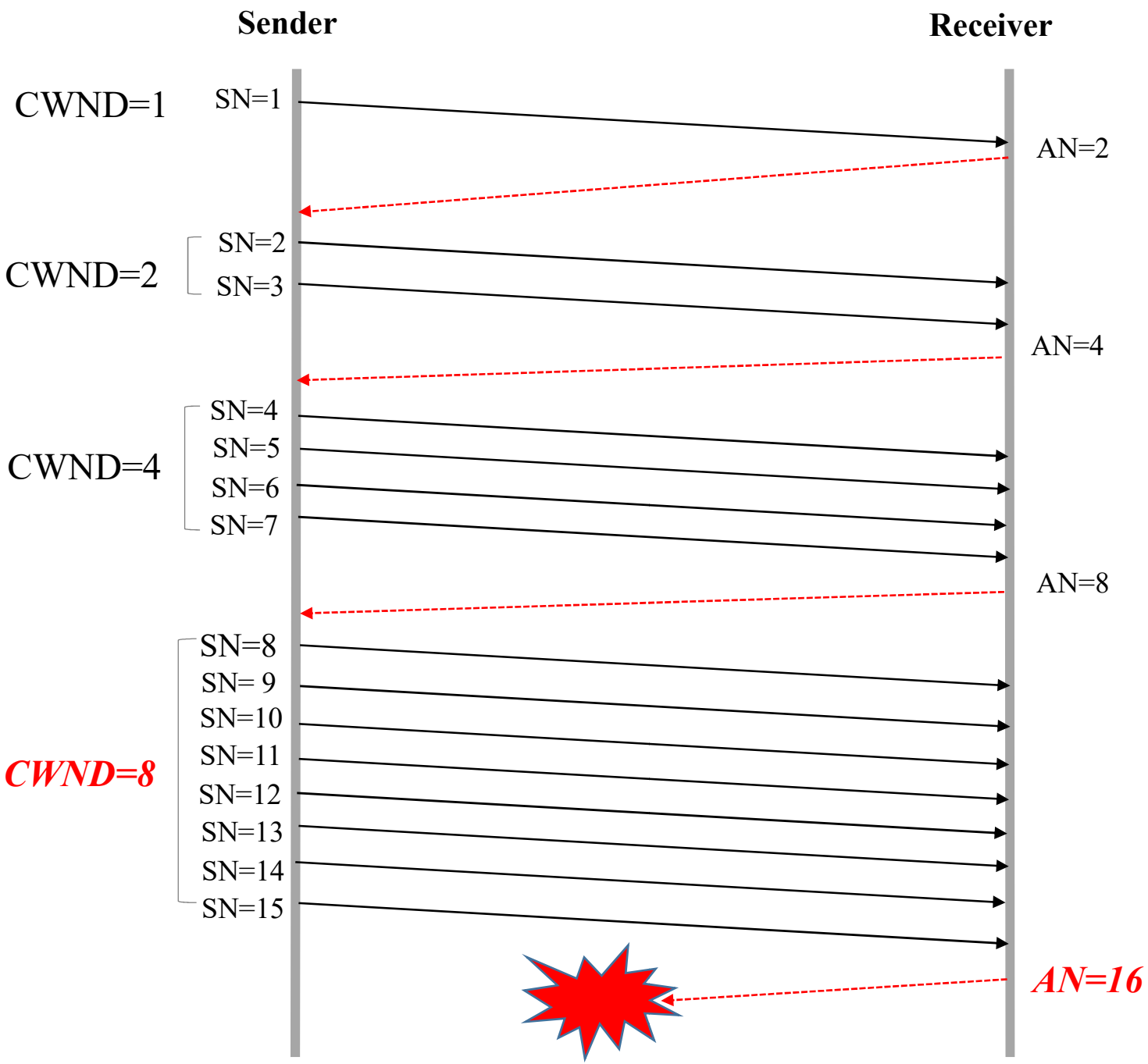
*** 빠른 복귀(fast recovery)**

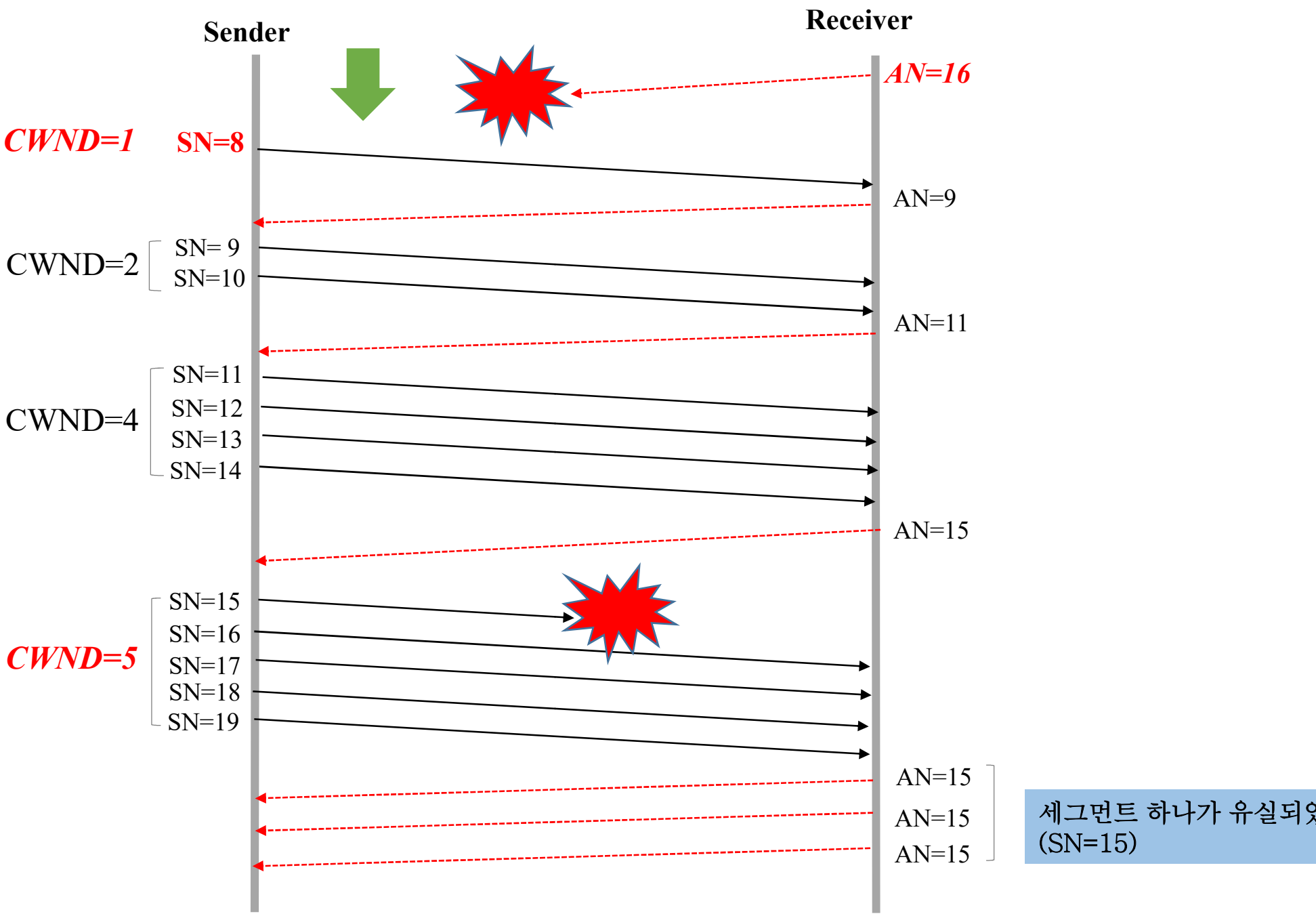
CWND의 크기를 1로 줄이지 않고 반(0.5)으로 줄임

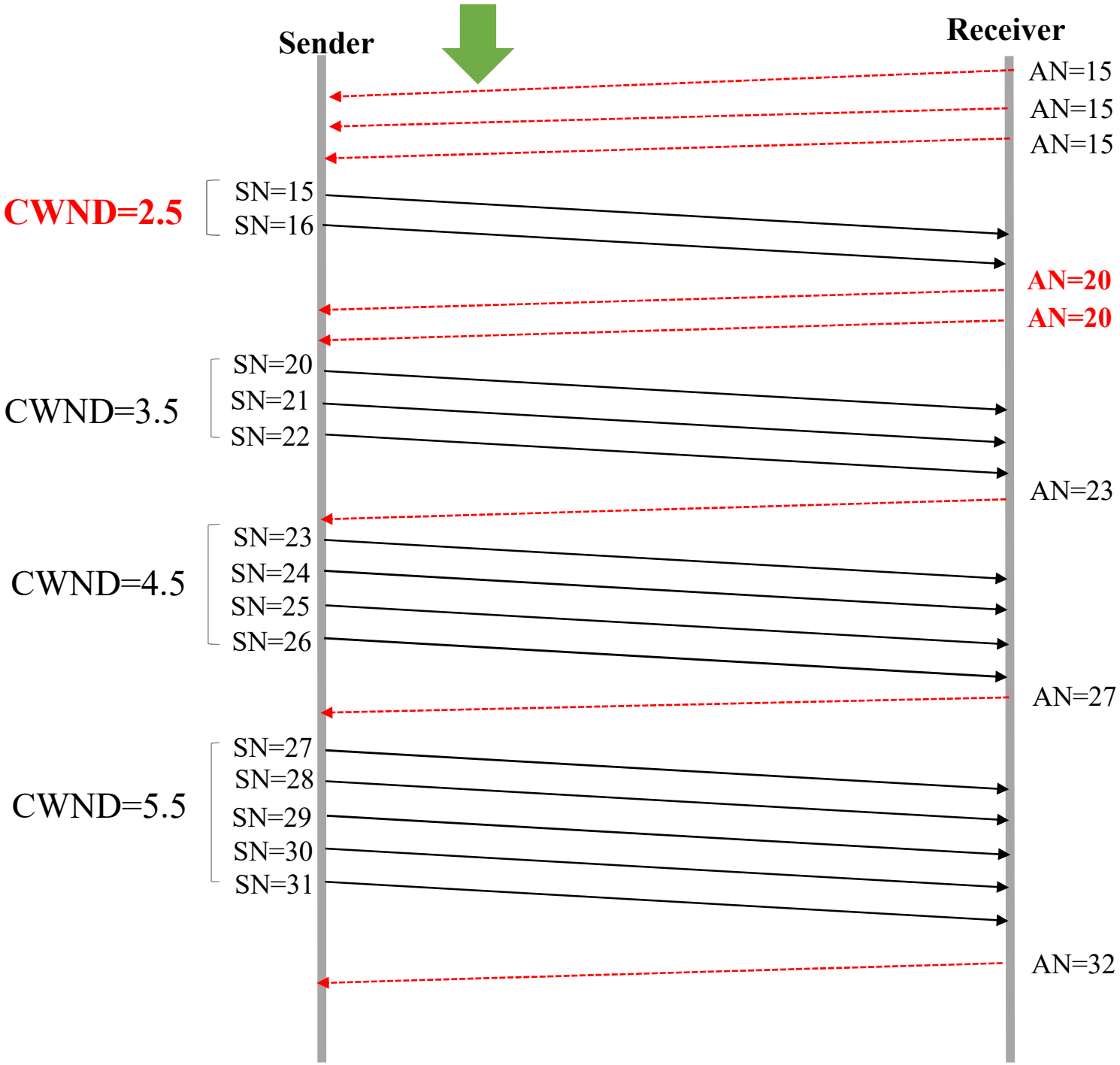


Fast Retransmission & Fast Recovery




Sender
Receiver
CWND=1
SN=1
AN=2
CWND=2
SN=2
SN=3
AN=4
CWND=4
SN=4
SN=5
SN=6
SN=7
AN=8
CWND=8
SN=8
SN= 9
SN=10
SN=11
SN=12
SN=13
SN=14
SN=15
AN=16


Sender
Receiver
AN=16
CWND=1
SN=8
AN=9
CWND=2
SN= 9
SN=10
AN=11
CWND=4
SN=11
SN=12
SN=13
SN=14
AN=15
CWND=5
SN=15
SN=16
SN=17
SN=18
SN=19
AN=15
AN=15
AN=15
세그먼트 하나가 유실되었
(SN=15)


Sender
Receiver
AN=15
AN=15
AN=15
CWND=2.5
SN=15
SN=16
AN=20
AN=20
CWND=3.5
SN=20
SN=21
SN=22
AN=23
CWND=4.5
SN=23
SN=24
SN=25
SN=26
AN=27
CWND=5.5
SN=27
SN=28
SN=29
SN=30
SN=31
AN=32